

C++ Bayesian Filtering Library

Generated by Doxygen 1.9.8

1 About this library	1
1.1 Dependencies	1
1.1.1 EIGEN	1
1.1.2 Doxygen (optional)	1
1.2 Demos	2
2 Namespace Index	5
2.1 Namespace List	5
3 Hierarchical Index	7
3.1 Class Hierarchy	7
4 Class Index	9
4.1 Class List	9
5 File Index	11
5.1 File List	11
6 Namespace Documentation	13
6.1 BayesianFiltering Namespace Reference	13
6.1.1 Typedef Documentation	14
6.1.1.1 Result	14
6.1.2 Function Documentation	14
6.1.2.1 MultivariateGaussianSampler()	14
6.1.2.2 MultivariateZeroMeanGaussianSampler()	15
6.1.2.3 ResultFormat() [1/2]	15
6.1.2.4 ResultFormat() [2/2]	15
6.1.2.5 UnivariateGaussianSampler()	16
6.1.2.6 UnivariateUniformSampler()	16
7 Class Documentation	17
7.1 BayesianFiltering::AdaptiveKalmanFilter Class Reference	17
7.1.1 Detailed Description	19
7.1.2 Constructor & Destructor Documentation	20
7.1.2.1 ~AdaptiveKalmanFilter()	20
7.1.3 Member Function Documentation	20
7.1.3.1 Correction()	20
7.1.3.2 Prediction()	20
7.1.3.3 Reset() [1/2]	20
7.1.3.4 Reset() [2/2]	21
7.1.4 Member Data Documentation	21
7.1.4.1 alpha	21
7.1.4.2 beta	21
7.2 BayesianFiltering::BayesianFilter Class Reference	22

7.2.1 Detailed Description	23
7.2.2 Constructor & Destructor Documentation	23
7.2.2.1 BayesianFilter()	23
7.2.2.2 ~BayesianFilter()	23
7.2.3 Member Function Documentation	23
7.2.3.1 Reset()	23
7.2.4 Member Data Documentation	24
7.2.4.1 d2_corrected	24
7.2.4.2 d2_predicted	24
7.2.4.3 P	24
7.2.4.4 S_corrected	24
7.2.4.5 S_predicted	24
7.2.4.6 V_corrected	24
7.2.4.7 V_predicted	25
7.2.4.8 X	25
7.2.4.9 Y_corrected	25
7.2.4.10 Y_predicted	25
7.3 BayesianFiltering::ExtendedKalmanFilter Class Reference	25
7.3.1 Detailed Description	27
7.3.2 Constructor & Destructor Documentation	27
7.3.2.1 ~ExtendedKalmanFilter()	27
7.3.3 Member Function Documentation	27
7.3.3.1 Correction()	27
7.3.3.2 Prediction()	28
7.3.3.3 Reset()	28
7.4 BayesianFiltering::KalmanFilter Class Reference	29
7.4.1 Detailed Description	31
7.4.2 Constructor & Destructor Documentation	31
7.4.2.1 ~KalmanFilter()	31
7.4.3 Member Function Documentation	31
7.4.3.1 Correction()	31
7.4.3.2 Prediction()	31
7.4.3.3 Reset()	32
7.5 BayesianFiltering::LinearBayesianFilter Class Reference	32
7.5.1 Detailed Description	34
7.5.2 Constructor & Destructor Documentation	34
7.5.2.1 ~LinearBayesianFilter()	34
7.5.3 Member Function Documentation	34
7.5.3.1 CheckModel()	34
7.5.3.2 Correction()	34
7.5.3.3 Prediction()	34
7.5.3.4 Reset()	35

7.6 BayesianFiltering::NonlinearBayesianFilter Class Reference	35
7.6.1 Detailed Description	36
7.6.2 Constructor & Destructor Documentation	37
7.6.2.1 ~NonlinearBayesianFilter()	37
7.6.3 Member Function Documentation	37
7.6.3.1 Correction()	37
7.6.3.2 Prediction()	37
7.6.3.3 Reset()	37
7.7 BayesianFiltering::RobustSelfAdaptiveKalmanFilter Class Reference	38
7.7.1 Detailed Description	40
7.7.2 Constructor & Destructor Documentation	40
7.7.2.1 ~RobustSelfAdaptiveKalmanFilter()	40
7.7.3 Member Function Documentation	40
7.7.3.1 Correction()	40
7.7.3.2 Prediction()	41
7.7.3.3 Reset()	41
7.8 BayesianFiltering::SamplingImportanceResamplingFilter Class Reference	41
7.8.1 Detailed Description	44
7.8.2 Member Enumeration Documentation	44
7.8.2.1 EstimateType	44
7.8.3 Constructor & Destructor Documentation	45
7.8.3.1 ~SamplingImportanceResamplingFilter()	45
7.8.4 Member Function Documentation	45
7.8.4.1 ComputeESS()	45
7.8.4.2 Correction()	45
7.8.4.3 EnumToString()	45
7.8.4.4 NormalizeWeights()	46
7.8.4.5 Prediction()	46
7.8.4.6 Reset() [1/2]	46
7.8.4.7 Reset() [2/2]	46
7.8.5 Member Data Documentation	47
7.8.5.1 ESS	47
7.8.5.2 estimate_type	47
7.8.5.3 Nparticles	47
7.8.5.4 Wparticles	47
7.8.5.5 Xparticles	47
7.8.5.6 Yparticles	47
7.9 BayesianFiltering::StateSpaceLinearModel Class Reference	48
7.9.1 Detailed Description	49
7.9.2 Member Function Documentation	49
7.9.2.1 MeasurementModel()	49
7.9.2.2 ProcessModel()	49

7.9.2.3 SetDimensions()	50
7.9.3 Member Data Documentation	50
7.9.3.1 A	50
7.9.3.2 B	50
7.9.3.3 C	50
7.9.3.4 Nu	51
7.9.3.5 Nv	51
7.9.3.6 Nw	51
7.9.3.7 Nx	51
7.9.3.8 Ny	51
7.9.3.9 P0	51
7.9.3.10 Q	51
7.9.3.11 R	51
7.9.3.12 X0	52
7.9.3.13 X_isangle	52
7.9.3.14 Y_isangle	52
7.10 BayesianFiltering::StateSpaceNonlinearModelBase Class Reference	52
7.10.1 Detailed Description	53
7.10.2 Constructor & Destructor Documentation	54
7.10.2.1 ~StateSpaceNonlinearModelBase()	54
7.10.3 Member Function Documentation	54
7.10.3.1 MeasurementModel()	54
7.10.3.2 ProcessModel()	54
7.10.3.3 SetDimensions()	55
7.10.3.4 UpdatedfdW()	55
7.10.3.5 UpdatedfdX()	55
7.10.3.6 UpdatedhdV()	56
7.10.3.7 UpdatedhdX()	56
7.10.4 Member Data Documentation	57
7.10.4.1 dfdW	57
7.10.4.2 dfdX	57
7.10.4.3 dhdV	57
7.10.4.4 dhdX	57
7.10.4.5 Nu	57
7.10.4.6 Nv	57
7.10.4.7 Nw	57
7.10.4.8 Nx	58
7.10.4.9 Ny	58
7.10.4.10 P0	58
7.10.4.11 Q	58
7.10.4.12 R	58
7.10.4.13 X0	58

7.10.4.14 X_isangle	58
7.10.4.15 Y_isangle	59
7.11 BayesianFiltering::UnscentedKalmanFilter Class Reference	59
7.11.1 Detailed Description	61
7.11.2 Constructor & Destructor Documentation	61
7.11.2.1 ~UnscentedKalmanFilter()	61
7.11.3 Member Function Documentation	62
7.11.3.1 Correction()	62
7.11.3.2 Prediction()	63
7.11.3.3 Reset()	63
8 File Documentation	65
8.1 bayesianfiltering.hpp File Reference	65
8.1.1 Macro Definition Documentation	67
8.1.1.1 PRINT_EIGEN_MATRIX	67
8.1.1.2 PRINT_EIGEN_VECTOR	67
8.1.1.3 PRINT_LINE_DEBUG	67
8.1.2 Function Documentation	68
8.1.2.1 PrintEigenMatrixXd()	68
8.1.2.2 PrintEigenVectorXd()	68
8.2 bayesianfiltering.hpp	68
8.3 README.md File Reference	82
Index	83

Chapter 1

About this library

This library is developed to be of simple use, requiring only knowledge on Bayesian filtering and object oriented C++ programming. It is contained in a single `bayesianfiltering.hpp` file.

It deals with both time varying linear and nonlinear discrete time models. For more details, see the documentation at `/doc` folder.

1.1 Dependencies

The library was developed with little dependence on other libraries or operating system. Some dependencies vary according to the demos provided. See the corresponding `Readme.md` file.

1.1.1 EIGEN

EIGEN is one of the most important libraries of linear algebra for C++. The Bayesian Filtering library relies strongly on it. Just install according to your system.

1.1.2 Doxygen (optional)

Doxygen is an optional installation, only if you want to update the documentation (html and/or pdf).

Installation can be done in different operational systems. See Doxygen project at <https://www.doxygen.nl/>

This library already has a Doxyfile configured at library root folder.

In the library root folder, run the following command:

```
$ doxygen
```

This will generate documentation in html pages (`doc/html`) and source files for latex/pdf version (`doc/latex`).

In `doc/html`, open `index.html` in any browser.

In `doc/latex`, in order to generate the pdf documentation just run:

```
$ make pdf
```

This will compile latex sources and generate `doc/latex/refman.pdf`

If latex and pdflatex are not installed in your system, install it.

For Linux, this can be accomplished by running:

```
$ sudo apt install texlive-latex-base
$ sudo apt install texlive-fonts-recommended texlive-latex-extra
```

Or, if you have lots of disk space, just run:

```
$ sudo apt install texlive-full
```

1.2 Demos

At `demos` folder you can find the current usage examples, with folders related to the kind of platform. Explore it! They are well documented.

For instance, Sampling Importance Resampling (SIR) particle filter demo with 1000 particles generate the following figures:

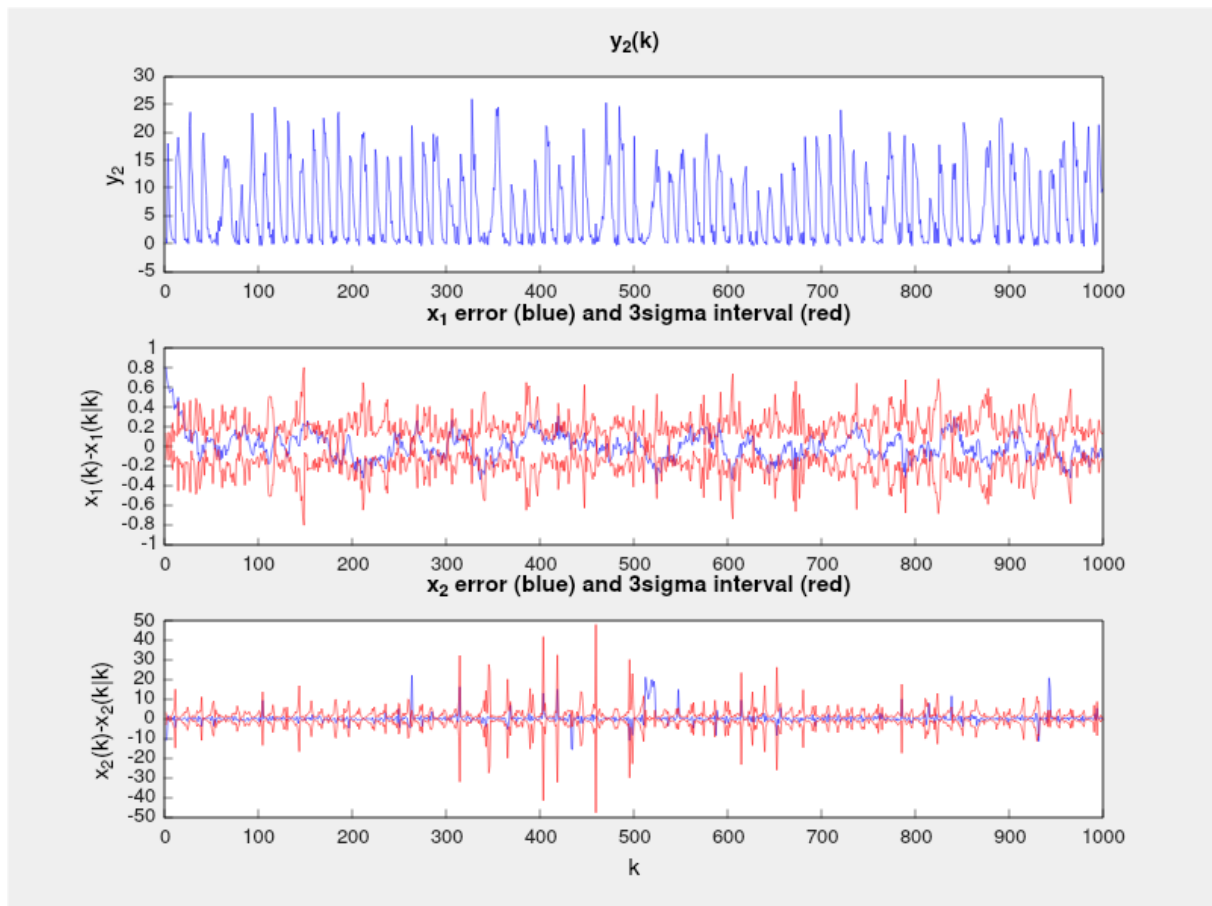


Figure 1.1 Figure 1

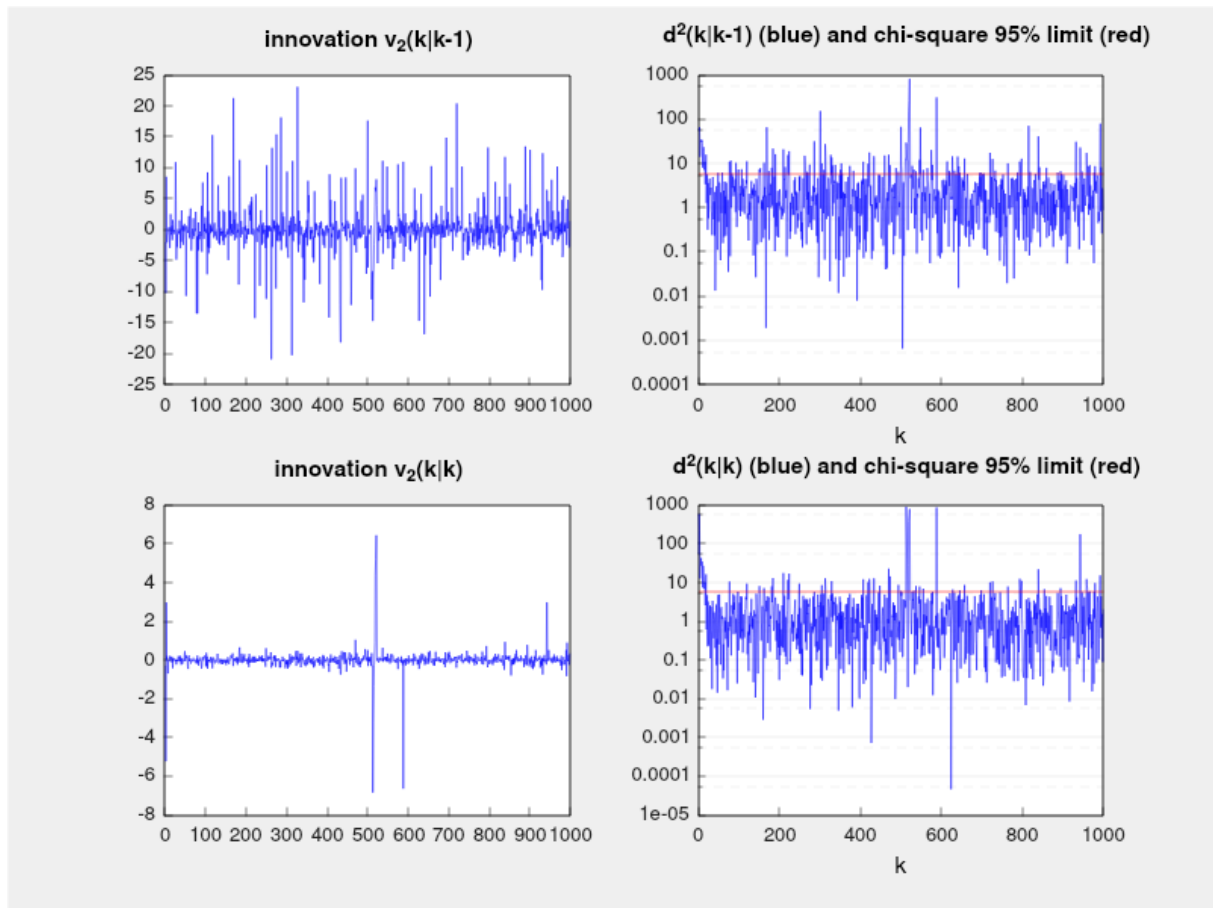


Figure 1.2 Figure 2

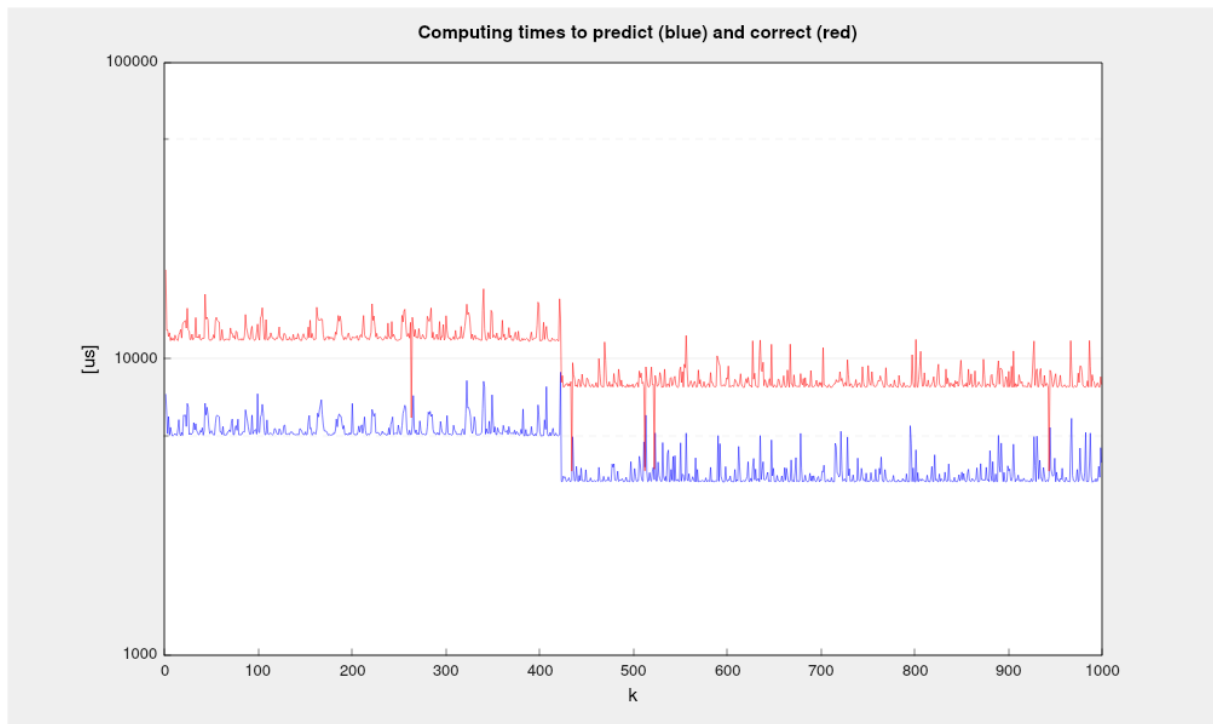


Figure 1.3 Figure 3

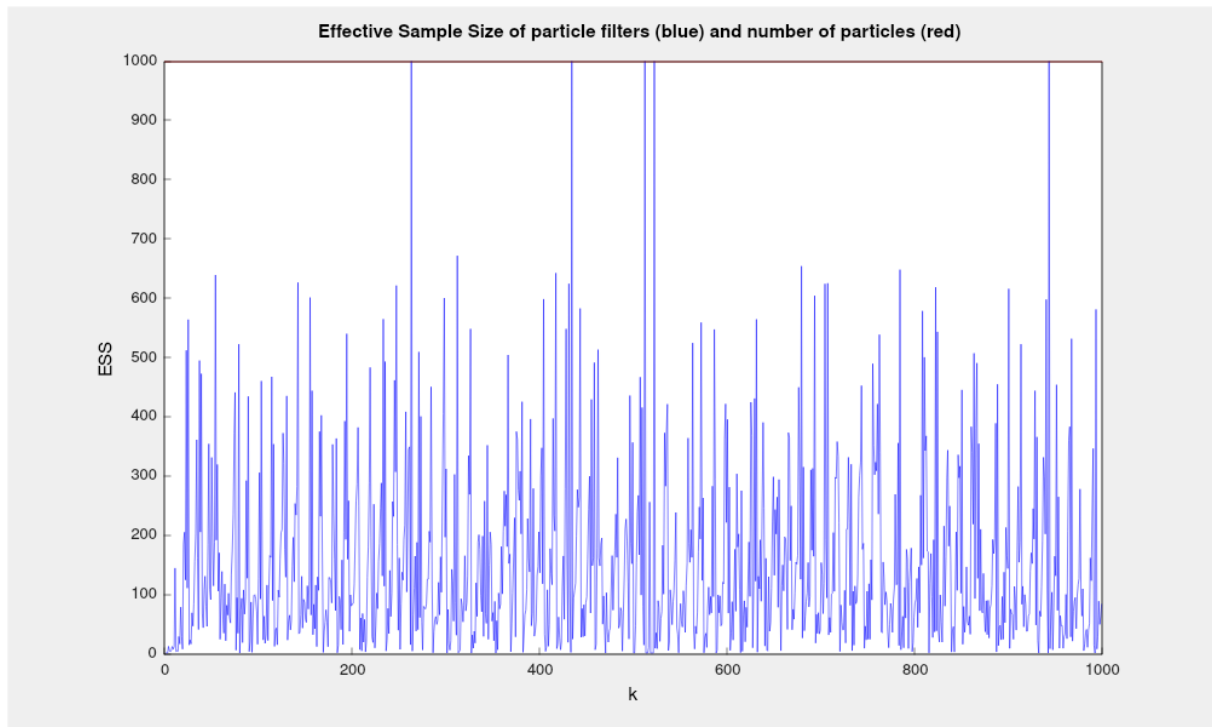


Figure 1.4 Figure 4

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all namespaces with brief descriptions:

BayesianFiltering	13
---	----

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

BayesianFiltering::BayesianFilter	22
BayesianFiltering::LinearBayesianFilter	32
BayesianFiltering::AdaptiveKalmanFilter	17
BayesianFiltering::KalmanFilter	29
BayesianFiltering::RobustSelfAdaptiveKalmanFilter	38
BayesianFiltering::NonlinearBayesianFilter	35
BayesianFiltering::ExtendedKalmanFilter	25
BayesianFiltering::SamplingImportanceResamplingFilter	41
BayesianFiltering::UnscentedKalmanFilter	59
BayesianFiltering::StateSpaceLinearModel	48
BayesianFiltering::StateSpaceNonlinearModelBase	52

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

BayesianFiltering::AdaptiveKalmanFilter	Adaptive Kalman filter, with $\mathbf{Q}$ and $\mathbf{R}$ matrices changing when every measurement comes according two user supplied learning parameters	17
BayesianFiltering::BayesianFilter	Base class of minimum variables of a Bayesian stochastic filter. All derived classes present filters for linear and nonlinear models. All filters follow the following loop	22
BayesianFiltering::ExtendedKalmanFilter	Extended Kalman filter, the first order approximation-based estimator for stochastic nonlinear state space models	25
BayesianFiltering::KalmanFilter	Kalman filter, the best optimal unbiased estimator for stochastic linear state space models . . .	29
BayesianFiltering::LinearBayesianFilter	Base class of Bayesian filters for state space linear models. See derived classes	32
BayesianFiltering::NonlinearBayesianFilter	Base class of Bayesian filters for state space nonlinear models. See derived classes	35
BayesianFiltering::RobustSelfAdaptiveKalmanFilter	Robust Self Adaptive Kalman filter, with $\mathbf{Q}$ and $\mathbf{R}$ matrices changing when every measurement comes using covariance matching strategy	38
BayesianFiltering::SamplingImportanceResamplingFilter	Sampling Importance Resampling (SIR) filter, particle-based estimator for stochastic nonlinear state space models	41
BayesianFiltering::StateSpaceLinearModel	Represents the parameters of a discrete time, generic linear state space model: 48	
BayesianFiltering::StateSpaceNonlinearModelBase	Represents the parameters of a discrete time, generic nonlinear state space model: 52	
BayesianFiltering::UnscentedKalmanFilter	Unscented Kalman filter, the Unscented Transform (UT)-based estimator for stochastic nonlinear state space models	59

Chapter 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

bayesianfiltering.hpp	65
---	----

Chapter 6

Namespace Documentation

6.1 BayesianFiltering Namespace Reference

Classes

- class [AdaptiveKalmanFilter](#)
Adaptive Kalman filter, with Q and R matrices changing when every measurement comes according two user supplied learning parameters.
- class [BayesianFilter](#)
Base class of minimum variables of a Bayesian stochastic filter. All derived classes present filters for linear and nonlinear models. All filters follow the following loop.
- class [ExtendedKalmanFilter](#)
Extended Kalman filter, the first order approximation-based estimator for stochastic nonlinear state space models.
- class [KalmanFilter](#)
Kalman filter, the best optimal unbiased estimator for stochastic linear state space models.
- class [LinearBayesianFilter](#)
Base class of Bayesian filters for state space linear models. See derived classes.
- class [NonlinearBayesianFilter](#)
Base class of Bayesian filters for state space nonlinear models. See derived classes.
- class [RobustSelfAdaptiveKalmanFilter](#)
Robust Self Adaptive Kalman filter, with Q and R matrices changing when every measurement comes using covariance matching strategy.
- class [SamplingImportanceResamplingFilter](#)
Sampling Importance Resampling (SIR) filter, particle-based estimator for stochastic nonlinear state space models.
- class [StateSpaceLinearModel](#)
Represents the parameters of a discrete time, generic linear state space model:
- class [StateSpaceNonlinearModelBase](#)
Represents the parameters of a discrete time, generic nonlinear state space model:
- class [UnscentedKalmanFilter](#)
Unscented Kalman filter, the Unscented Transform (UT)-based estimator for stochastic nonlinear state space models.

Typedefs

- typedef std::pair< bool, std::string > [Result](#)
Encapsulated <bool,std::string> pair response of a function, with .first specifying status (success of failure) and .second the reason string in case of failure.

Functions

- [Result ResultFormat](#) (bool success, const std::string &message)
Format <bool,std::string> pair response with defined status and reason string.
- `template<typename ... Args>`
[Result ResultFormat](#) (bool success, const std::string &format, Args ... args)
Format <bool,std::string> pair response with defined status and reason string to be printf-like formatted.
- [Result MultivariateZeroMeanGaussianSampler](#) (Eigen::VectorXd &sample, Eigen::MatrixXd &covariance_↵
matrix)
Generate multivariate $sample \sim N(0, covariance_matrix)$.
- [Result MultivariateGaussianSampler](#) (Eigen::VectorXd &sample, Eigen::VectorXd &mean_vector, Eigen::↵
MatrixXd &covariance_matrix)
Generate multivariate $sample \sim N(mean_vector, covariance_matrix)$.
- double [UnivariateGaussianSampler](#) (double mean, double variance)
Generate univariate $sample \sim N(mean, variance)$.
- double [UnivariateUniformSampler](#) (double min_value, double max_value)
Generate univariate $sample \sim U(min_value, max_value)$.

6.1.1 Typedef Documentation

6.1.1.1 Result

```
typedef std::pair<bool, std::string> BayesianFiltering::Result
```

Encapsulated <bool,std::string> pair response of a function, with .first specifying status (success of failure) and .second the reason string in case of failure.

6.1.2 Function Documentation

6.1.2.1 MultivariateGaussianSampler()

```
Result BayesianFiltering::MultivariateGaussianSampler (
    Eigen::VectorXd & sample,
    Eigen::VectorXd & mean_vector,
    Eigen::MatrixXd & covariance_matrix )
```

Generate multivariate $sample \sim N(mean_vector, covariance_matrix)$.

Parameters

out	<i>sample</i>	will be filled with the outcome
in	<i>mean_vector</i>	as above
in	<i>covariance_matrix</i>	as above

Returns

Result type

6.1.2.2 MultivariateZeroMeanGaussianSampler()

```
Result BayesianFiltering::MultivariateZeroMeanGaussianSampler (
    Eigen::VectorXd & sample,
    Eigen::MatrixXd & covariance_matrix )
```

Generate multivariate sample $\sim N(0, \text{covariance_matrix})$.

Parameters

out	<i>sample</i>	will be filled with the outcome
in	<i>covariance_matrix</i>	as above

Returns

Result type

6.1.2.3 ResultFormat() [1/2]

```
template<typename ... Args>
Result BayesianFiltering::ResultFormat (
    bool success,
    const std::string & format,
    Args ... args )
```

Format <bool,std::string> pair response with defined status and reason string to be printf-like formatted.

Parameters

in	<i>success</i>	bool indicating status
in	<i>format</i>	formatting string
in	<i>args</i>	sequence of variable arguments for message formatting

6.1.2.4 ResultFormat() [2/2]

```
Result BayesianFiltering::ResultFormat (
    bool success,
    const std::string & message ) [inline]
```

Format <bool,std::string> pair response with defined status and reason string.

Parameters

in	<i>success</i>	bool indicating status
in	<i>message</i>	string message

6.1.2.5 UnivariateGaussianSampler()

```
double BayesianFiltering::UnivariateGaussianSampler (
    double mean,
    double variance )
```

Generate univariate $\text{sample} \sim \mathcal{N}(\text{mean}, \text{variance})$.

Parameters

in	<i>mean</i>	as above
in	<i>variance</i>	as above

Returns

sample

6.1.2.6 UnivariateUniformSampler()

```
double BayesianFiltering::UnivariateUniformSampler (
    double min_value,
    double max_value )
```

Generate univariate $\text{sample} \sim \mathcal{U}(\text{min_value}, \text{max_value})$.

Parameters

in	<i>min_value</i>	as above
in	<i>max_value</i>	as above

Returns

sample

Chapter 7

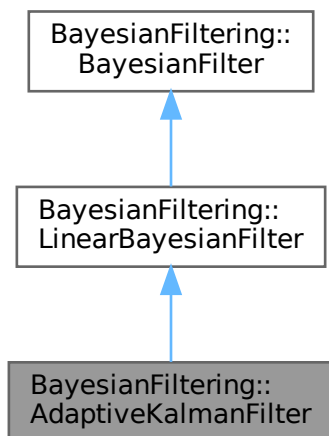
Class Documentation

7.1 BayesianFiltering::AdaptiveKalmanFilter Class Reference

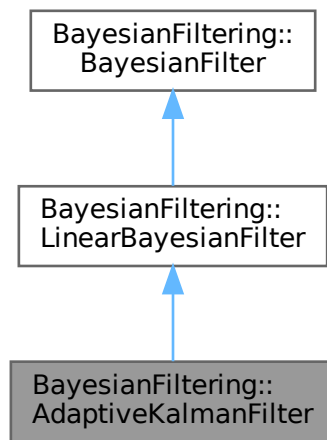
Adaptive Kalman filter, with Q and R matrices changing when every measurement comes according two user supplied learning parameters.

```
#include <bayesianfiltering.hpp>
```

Inheritance diagram for BayesianFiltering::AdaptiveKalmanFilter:



Collaboration diagram for BayesianFiltering::AdaptiveKalmanFilter:



Public Member Functions

- virtual [~AdaptiveKalmanFilter](#) ()
- void [Reset](#) ([StateSpaceLinearModel](#) &linear_model, double _alpha, double _beta)
Reset filter with user supplied filter specific parameters. Must be called before the prediction/correction cycle.
- void [Reset](#) ([StateSpaceLinearModel](#) &linear_model) override
Reset filter with default filter specific parameters ($\alpha = 0.95$ and $\beta = 0.99$). Must be called before the prediction/correction cycle.
- [Result Prediction](#) ([Eigen::VectorXd](#) &U, [StateSpaceLinearModel](#) &linear_model) override
Prediction procedure, which will update (in place) the internal member variables X and P based on input vector U and model $linear_model$. In case of time varying systems, $linear_model$ class members should be updated before calling this procedure.
- [Result Correction](#) ([Eigen::VectorXd](#) &Y, [StateSpaceLinearModel](#) &linear_model) override
Correction procedure, which will update (in place) the internal member variables X and P based on measurement vector Y and model $linear_model$. In case of time varying systems, $linear_model$ class members should be updated before calling this procedure.

Public Member Functions inherited from [BayesianFiltering::LinearBayesianFilter](#)

- virtual [~LinearBayesianFilter](#) ()

Public Member Functions inherited from [BayesianFiltering::BayesianFilter](#)

- [BayesianFilter](#) ()
- virtual [~BayesianFilter](#) ()

Public Attributes

- double [alpha](#)
User supplied learning parameter for updating R measurement noise covariance matrix. Must be in interval $]0,1]$, with 1 leading to fixed R matrix.
- double [beta](#)
User supplied learning parameter for updating Q process noise covariance matrix. Must be in interval $]0,1]$, with 1 leading to fixed Q matrix.

Public Attributes inherited from [BayesianFiltering::BayesianFilter](#)

- Eigen::VectorXd [X](#)
State vector predictor $\hat{x}(k|k-1)$ or estimate $\hat{x}(k|k)$, depending on which stage the filter is.
- Eigen::MatrixXd [P](#)
State vector error covariance matrix predictor $P(k|k-1)$ or $P(k|k)$, depending on which stage the filter is.
- Eigen::VectorXd [Y_predicted](#)
Predicted measurement $\hat{y}(k|k-1)$.
- Eigen::VectorXd [V_predicted](#)
Predicted innovation $y(k) - \hat{y}(k|k-1)$.
- Eigen::MatrixXd [S_predicted](#)
Predicted innovation covariance matrix.
- double [d2_predicted](#)
Predicted innovation Mahalanobis distance, allowing to verify a filter as optimistic or pessimistic, and divergence.
- Eigen::VectorXd [Y_corrected](#)
Corrected measurement $\hat{y}(k|k)$.
- Eigen::VectorXd [V_corrected](#)
Corrected innovation $y(k) - \hat{y}(k|k)$.
- Eigen::MatrixXd [S_corrected](#)
Corrected innovation covariance matrix.
- double [d2_corrected](#)
Corrected innovation Mahalanobis distance, allowing to verify a filter as optimistic or pessimistic, and divergence.

Additional Inherited Members**Protected Member Functions inherited from [BayesianFiltering::LinearBayesianFilter](#)**

- [Result CheckModel](#) ([StateSpaceLinearModel](#) &linear_model)

Protected Member Functions inherited from [BayesianFiltering::BayesianFilter](#)

- void [Reset](#) (Eigen::VectorXd &X0, Eigen::MatrixXd &P0, int Nx, int Ny)
Reset filter, first procedure before prediction/correction.

7.1.1 Detailed Description

Adaptive Kalman filter, with Q and R matrices changing when every measurement comes according two user supplied learning parameters.

This implementation is specific for discrete time, continuous state models.

7.1.2 Constructor & Destructor Documentation

7.1.2.1 ~AdaptiveKalmanFilter()

```
virtual BayesianFiltering::AdaptiveKalmanFilter::~AdaptiveKalmanFilter ( ) [inline], [virtual]
```

7.1.3 Member Function Documentation

7.1.3.1 Correction()

```
Result BayesianFiltering::AdaptiveKalmanFilter::Correction (
    Eigen::VectorXd & Y,
    StateSpaceLinearModel & linear_model ) [inline], [override], [virtual]
```

Correction procedure, which will update (in place) the internal member variables X and P based on measurement vector Y and model `linear_model`. In case of time varying systems, `linear_model` class members should be updated before calling this procedure.

Parameters

in	<i>Y</i>	measurement vector of the linear system process model
in	<i>linear_model</i>	StateSpaceLinearModel object for which the filter will run

Implements [BayesianFiltering::LinearBayesianFilter](#).

7.1.3.2 Prediction()

```
Result BayesianFiltering::AdaptiveKalmanFilter::Prediction (
    Eigen::VectorXd & U,
    StateSpaceLinearModel & linear_model ) [inline], [override], [virtual]
```

Prediction procedure, which will update (in place) the internal member variables X and P based on input vector U and model `linear_model`. In case of time varying systems, `linear_model` class members should be updated before calling this procedure.

Parameters

in	<i>U</i>	input vector of the linear system process model
in	<i>linear_model</i>	StateSpaceLinearModel object for which the filter will run

Implements [BayesianFiltering::LinearBayesianFilter](#).

7.1.3.3 Reset() [1/2]

```
void BayesianFiltering::AdaptiveKalmanFilter::Reset (
    StateSpaceLinearModel & linear_model ) [inline], [override], [virtual]
```

Reset filter with default filter specific parameters (`alpha = 0.95` and `beta = 0.99`). Must be called before the prediction/correction cycle.

Parameters

in	<i>linear_model</i>	StateSpaceLinearModel object for which the filter will run
----	---------------------	--

Implements [BayesianFiltering::LinearBayesianFilter](#).

7.1.3.4 Reset() [2/2]

```
void BayesianFiltering::AdaptiveKalmanFilter::Reset (
    StateSpaceLinearModel & linear_model,
    double _alpha,
    double _beta ) [inline]
```

Reset filter with user supplied filter specific parameters. Must be called before the prediction/correction cycle.

Parameters

in	<i>linear_model</i>	StateSpaceLinearModel object for which the filter will run
in	<i>_alpha</i>	user provided value for alpha member variable
in	<i>_beta</i>	user provided value for beta member variable

7.1.4 Member Data Documentation

7.1.4.1 alpha

```
double BayesianFiltering::AdaptiveKalmanFilter::alpha
```

User supplied learning parameter for updating R measurement noise covariance matrix. Must be in interval $]0,1]$, with 1 leading to fixed R matrix.

7.1.4.2 beta

```
double BayesianFiltering::AdaptiveKalmanFilter::beta
```

User supplied learning parameter for updating Q process noise covariance matrix. Must be in interval $]0,1]$, with 1 leading to fixed Q matrix.

The documentation for this class was generated from the following file:

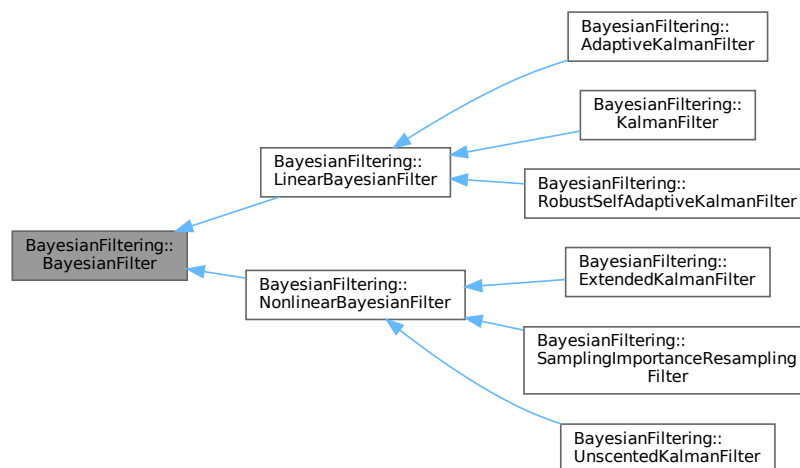
- [bayesianfiltering.hpp](#)

7.2 BayesianFiltering::BayesianFilter Class Reference

Base class of minimum variables of a Bayesian stochastic filter. All derived classes present filters for linear and nonlinear models. All filters follow the following loop.

```
#include <bayesianfiltering.hpp>
```

Inheritance diagram for BayesianFiltering::BayesianFilter:



Public Member Functions

- [BayesianFilter](#) ()
- virtual [~BayesianFilter](#) ()

Public Attributes

- Eigen::VectorXd [X](#)
State vector predictor $\hat{x}(k|k-1)$ or estimate $\hat{x}(k|k)$, depending on which stage the filter is.
- Eigen::MatrixXd [P](#)
State vector error covariance matrix predictor $P(k|k-1)$ or $P(k|k)$, depending on which stage the filter is.
- Eigen::VectorXd [Y_predicted](#)
Predicted measurement $\hat{y}(k|k-1)$.
- Eigen::VectorXd [V_predicted](#)
Predicted innovation $y(k) - \hat{y}(k|k-1)$.
- Eigen::MatrixXd [S_predicted](#)
Predicted innovation covariance matrix.
- double [d2_predicted](#)
Predicted innovation Mahalanobis distance, allowing to verify a filter as optimistic or pessimistic, and divergence.
- Eigen::VectorXd [Y_corrected](#)
Corrected measurement $\hat{y}(k|k)$.
- Eigen::VectorXd [V_corrected](#)
Corrected innovation $y(k) - \hat{y}(k|k)$.
- Eigen::MatrixXd [S_corrected](#)
Corrected innovation covariance matrix.
- double [d2_corrected](#)
Corrected innovation Mahalanobis distance, allowing to verify a filter as optimistic or pessimistic, and divergence.

Protected Member Functions

- void [Reset](#) (Eigen::VectorXd &X0, Eigen::MatrixXd &P0, int Nx, int Ny)
Reset filter, first procedure before prediction/correction.

7.2.1 Detailed Description

Base class of minimum variables of a Bayesian stochastic filter. All derived classes present filters for linear and nonlinear models. All filters follow the following loop.

1. Reset
2. Prediction: compute $x(k|k-1)$ using $x(k-1)$ and input $u(k)$
3. Correction, in case of measurement available: compute $x(k|k)$ using $x(k|k-1)$ and measurement vector $y(k)$;
4. In case of filter divergence, perform necessary actions and go back to step 1. Otherwise, go back to step 2.

As it can be seen, as described above correction is optional, which allows filter continuing by prediction. Internal member variables present the current filter state.

System model object should be not used in other purposes other than to be used as input to the filter. For the case of linear time varying models, it can have their parameters directly updated just before calling prediction/correction methods, as a way to deal with time varying systems. For the case of nonlinear time varying models, parameters updating is done in derived model class, where the methods are called from inside filter object. Thus, model updating is done in derived class, being not necessary to be done by the user.

7.2.2 Constructor & Destructor Documentation

7.2.2.1 BayesianFilter()

```
BayesianFiltering::BayesianFilter::BayesianFilter ( ) [inline]
```

7.2.2.2 ~BayesianFilter()

```
virtual BayesianFiltering::BayesianFilter::~~BayesianFilter ( ) [inline], [virtual]
```

7.2.3 Member Function Documentation

7.2.3.1 Reset()

```
void BayesianFiltering::BayesianFilter::Reset (
    Eigen::VectorXd & X0,
    Eigen::MatrixXd & P0,
    int Nx,
    int Ny ) [inline], [protected]
```

Reset filter, first procedure before prediction/correction.

Parameters

in	$X0$	initial state vector mean value of $x(0)$ distribution
in	$P0$	initial state error covariance matrix of $x(0)$ distribution
in	Nx	Number of state variables in vector X
in	Ny	Number of measurements in vector Y

7.2.4 Member Data Documentation**7.2.4.1 d2_corrected**

```
double BayesianFiltering::BayesianFilter::d2_corrected
```

Corrected innovation Mahalanobis distance, allowing to verify a filter os optimistic ou pessimistic, and divergence.

7.2.4.2 d2_predicted

```
double BayesianFiltering::BayesianFilter::d2_predicted
```

Predicted innovation Mahalanobis distance, allowing to verify a filter os optimistic ou pessimistic, and divergence.

7.2.4.3 P

```
Eigen::MatrixXd BayesianFiltering::BayesianFilter::P
```

State vector error covariance matrix predictor $P(k|k-1)$ or $P(k|k)$, depending on which stage the filter is.

7.2.4.4 S_corrected

```
Eigen::MatrixXd BayesianFiltering::BayesianFilter::S_corrected
```

Corrected innovation covariance matrix.

7.2.4.5 S_predicted

```
Eigen::MatrixXd BayesianFiltering::BayesianFilter::S_predicted
```

Predicted innovation covariance matrix.

7.2.4.6 V_corrected

```
Eigen::VectorXd BayesianFiltering::BayesianFilter::V_corrected
```

Corrected innovation $y(k) - y(k|k)$.

7.2.4.7 V_predicted

`Eigen::VectorXd BayesianFiltering::BayesianFilter::V_predicted`

Predicted innovation $\hat{y}(k) - \hat{y}(k|k-1)$.

7.2.4.8 X

`Eigen::VectorXd BayesianFiltering::BayesianFilter::X`

State vector predictor $\hat{x}(k|k-1)$ or estimate $\hat{x}(k|k)$, depending on which stage the filter is.

7.2.4.9 Y_corrected

`Eigen::VectorXd BayesianFiltering::BayesianFilter::Y_corrected`

Corrected measurement $y(k|k)$.

7.2.4.10 Y_predicted

`Eigen::VectorXd BayesianFiltering::BayesianFilter::Y_predicted`

Predicted measurement $\hat{y}(k|k-1)$.

The documentation for this class was generated from the following file:

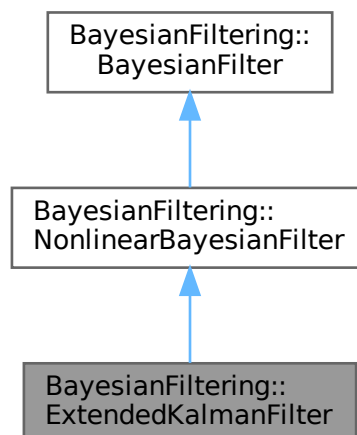
- [bayesianfiltering.hpp](#)

7.3 BayesianFiltering::ExtendedKalmanFilter Class Reference

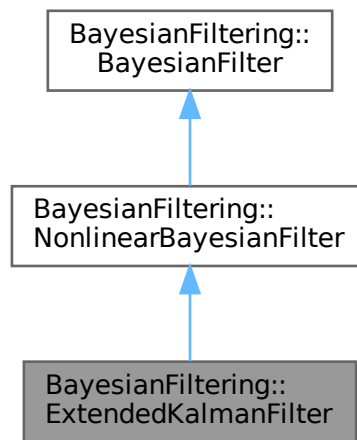
Extended Kalman filter, the first order approximation-based estimator for stochastic nonlinear state space models.

```
#include <bayesianfiltering.hpp>
```

Inheritance diagram for BayesianFiltering::ExtendedKalmanFilter:



Collaboration diagram for BayesianFiltering::ExtendedKalmanFilter:



Public Member Functions

- virtual [~ExtendedKalmanFilter](#) ()
- void [Reset](#) ([StateSpaceNonlinearModelBase](#) &nonlinear_model) override
Reset filter, first procedure before prediction/correction.
- [Result Prediction](#) ([Eigen::VectorXd](#) &U, double t, [StateSpaceNonlinearModelBase](#) &nonlinear_model) override
Prediction procedure, which will update (in place) the internal member variables X and P based on input vector U and model `nonlinear_model`. This procedure calls methods of the derived class object `nonlinear_model` in order to update model parameters. So there is no need the user to do that.
- [Result Correction](#) ([Eigen::VectorXd](#) &Y, double t, [StateSpaceNonlinearModelBase](#) &nonlinear_model) override
Correction procedure, which will update (in place) the internal member variables X and P based on measurement vector Y and model `linear_model`. In case of time varying systems, `linear_model` class members should be updated before calling this procedure.

Public Member Functions inherited from [BayesianFiltering::NonlinearBayesianFilter](#)

- virtual [~NonlinearBayesianFilter](#) ()

Public Member Functions inherited from [BayesianFiltering::BayesianFilter](#)

- [BayesianFilter](#) ()
- virtual [~BayesianFilter](#) ()

Additional Inherited Members

Public Attributes inherited from BayesianFiltering::BayesianFilter

- Eigen::VectorXd [X](#)
State vector predictor $\hat{x}(k|k-1)$ or estimate $\hat{x}(k|k)$, depending on which stage the filter is.
- Eigen::MatrixXd [P](#)
State vector error covariance matrix predictor $P(k|k-1)$ or $P(k|k)$, depending on which stage the filter is.
- Eigen::VectorXd [Y_predicted](#)
Predicted measurement $\hat{y}(k|k-1)$.
- Eigen::VectorXd [V_predicted](#)
Predicted innovation $y(k) - \hat{y}(k|k-1)$.
- Eigen::MatrixXd [S_predicted](#)
Predicted innovation covariance matrix.
- double [d2_predicted](#)
Predicted innovation Mahalanobis distance, allowing to verify a filter as optimistic ou pessimistic, and divergence.
- Eigen::VectorXd [Y_corrected](#)
Corrected measurement $\hat{y}(k|k)$.
- Eigen::VectorXd [V_corrected](#)
Corrected innovation $y(k) - \hat{y}(k|k)$.
- Eigen::MatrixXd [S_corrected](#)
Corrected innovation covariance matrix.
- double [d2_corrected](#)
Corrected innovation Mahalanobis distance, allowing to verify a filter as optimistic ou pessimistic, and divergence.

Protected Member Functions inherited from BayesianFiltering::BayesianFilter

- void [Reset](#) (Eigen::VectorXd &X0, Eigen::MatrixXd &P0, int Nx, int Ny)
Reset filter, first procedure before prediction/correction.

7.3.1 Detailed Description

Extended Kalman filter, the first order approximation-based estimator for stochastic nonlinear state space models. This implementation is specific for discrete time, continuous state models.

7.3.2 Constructor & Destructor Documentation

7.3.2.1 ~ExtendedKalmanFilter()

```
virtual BayesianFiltering::ExtendedKalmanFilter::~ExtendedKalmanFilter ( ) [inline], [virtual]
```

7.3.3 Member Function Documentation

7.3.3.1 Correction()

```
Result BayesianFiltering::ExtendedKalmanFilter::Correction (
    Eigen::VectorXd & Y,
    double t,
    StateSpaceNonlinearModelBase & nonlinear_model ) [inline], [override], [virtual]
```

Correction procedure, which will update (in place) the internal member variables X and P based on measurement vector Y and model `linear_model`. In case of time varying systems, `linear_model` class members should be updated before calling this procedure.

Parameters

in	Y	measurement vector of the linear system process model
in	t	time variable, for time varying systems
in	<i>nonlinear_model</i>	StateSpaceNonlinearModelBase derived object for which the filter will run. Model updating and parameter calculations implemented by the user in the StateSpaceNonlinearModelBase derived object are called from inside this method.

Implements [BayesianFiltering::NonlinearBayesianFilter](#).

7.3.3.2 Prediction()

```
Result BayesianFiltering::ExtendedKalmanFilter::Prediction (
    Eigen::VectorXd & U,
    double t,
    StateSpaceNonlinearModelBase & nonlinear_model ) [inline], [override], [virtual]
```

Prediction procedure, which will update (in place) the internal member variables X and P based on input vector U and model *nonlinear_model*. This procedure calls methods of the derived class object *nonlinear_model* in order to update model parameters. So there is no need the user to do that.

Parameters

in	U	input vector of the linear system process model
in	t	time variable, for time varying systems
in	<i>nonlinear_model</i>	StateSpaceNonlinearModelBase derived object for which the filter will run. Model updating and parameter calculations implemented by the user in the StateSpaceNonlinearModelBase derived object are called from inside this method.

Implements [BayesianFiltering::NonlinearBayesianFilter](#).

7.3.3.3 Reset()

```
void BayesianFiltering::ExtendedKalmanFilter::Reset (
    StateSpaceNonlinearModelBase & nonlinear_model ) [inline], [override], [virtual]
```

Reset filter, first procedure before prediction/correction.

Parameters

in	<i>nonlinear_model</i>	StateSpaceNonlinearModelBase derived object for which the filter will run
----	------------------------	---

Implements [BayesianFiltering::NonlinearBayesianFilter](#).

The documentation for this class was generated from the following file:

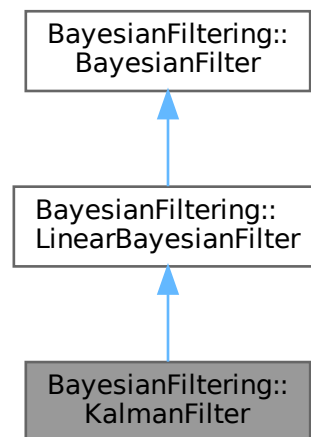
- [bayesianfiltering.hpp](#)

7.4 BayesianFiltering::KalmanFilter Class Reference

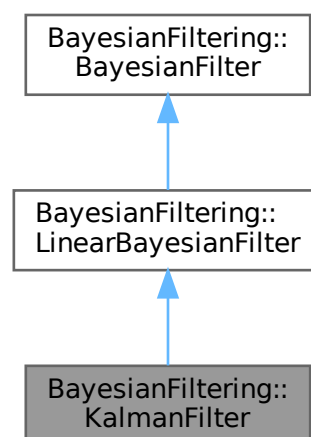
Kalman filter, the best optimal unbiased estimator for stochastic linear state space models.

```
#include <bayesianfiltering.hpp>
```

Inheritance diagram for BayesianFiltering::KalmanFilter:



Collaboration diagram for BayesianFiltering::KalmanFilter:



Public Member Functions

- virtual [~KalmanFilter](#) ()
- void [Reset](#) ([StateSpaceLinearModel](#) &linear_model) override
Reset filter, first procedure before prediction/correction.
- [Result Prediction](#) ([Eigen::VectorXd](#) &U, [StateSpaceLinearModel](#) &linear_model) override
Prediction procedure, which will update (in place) the internal member variables X and P based on input vector U and model `linear_model`. In case of time varying systems, `linear_model` class members should be updated before calling this procedure.
- [Result Correction](#) ([Eigen::VectorXd](#) &Y, [StateSpaceLinearModel](#) &linear_model) override
Correction procedure, which will update (in place) the internal member variables X and P based on measurement vector Y and model `linear_model`. In case of time varying systems, `linear_model` class members should be updated before calling this procedure.

Public Member Functions inherited from [BayesianFiltering::LinearBayesianFilter](#)

- virtual [~LinearBayesianFilter](#) ()

Public Member Functions inherited from [BayesianFiltering::BayesianFilter](#)

- [BayesianFilter](#) ()
- virtual [~BayesianFilter](#) ()

Additional Inherited Members

Public Attributes inherited from [BayesianFiltering::BayesianFilter](#)

- [Eigen::VectorXd](#) [X](#)
State vector predictor $x(k|k-1)$ or estimate $x(k|k)$, depending on which stage the filter is.
- [Eigen::MatrixXd](#) [P](#)
State vector error covariance matrix predictor $P(k|k-1)$ or $P(k|k)$, depending on which stage the filter is.
- [Eigen::VectorXd](#) [Y_predicted](#)
Predicted measurement $y(k|k-1)$.
- [Eigen::VectorXd](#) [V_predicted](#)
Predicted innovation $y(k) - y(k|k-1)$.
- [Eigen::MatrixXd](#) [S_predicted](#)
Predicted innovation covariance matrix.
- double [d2_predicted](#)
Predicted innovation Mahalanobis distance, allowing to verify a filter as optimistic ou pessimistic, and divergence.
- [Eigen::VectorXd](#) [Y_corrected](#)
Corrected measurement $y(k|k)$.
- [Eigen::VectorXd](#) [V_corrected](#)
Corrected innovation $y(k) - y(k|k)$.
- [Eigen::MatrixXd](#) [S_corrected](#)
Corrected innovation covariance matrix.
- double [d2_corrected](#)
Corrected innovation Mahalanobis distance, allowing to verify a filter as optimistic ou pessimistic, and divergence.

Protected Member Functions inherited from [BayesianFiltering::LinearBayesianFilter](#)

- [Result CheckModel](#) ([StateSpaceLinearModel](#) &linear_model)

Protected Member Functions inherited from [BayesianFiltering::BayesianFilter](#)

- void [Reset](#) (Eigen::VectorXd &X0, Eigen::MatrixXd &P0, int Nx, int Ny)
Reset filter, first procedure before prediction/correction.

7.4.1 Detailed Description

Kalman filter, the best optimal unbiased estimator for stochastic linear state space models.

This implementation is specific for discrete time, continuous state models.

7.4.2 Constructor & Destructor Documentation

7.4.2.1 ~KalmanFilter()

```
virtual BayesianFiltering::KalmanFilter::~~KalmanFilter ( ) [inline], [virtual]
```

7.4.3 Member Function Documentation

7.4.3.1 Correction()

```
Result BayesianFiltering::KalmanFilter::Correction (
    Eigen::VectorXd & Y,
    StateSpaceLinearModel & linear_model ) [inline], [override], [virtual]
```

Correction procedure, which will update (in place) the internal member variables X and P based on measurement vector Y and model linear_model. In case of time varying systems, linear_model class members should be updated before calling this procedure.

Parameters

in	Y	measurement vector of the linear system process model
in	linear_model	StateSpaceLinearModel object for which the filter will run

Implements [BayesianFiltering::LinearBayesianFilter](#).

7.4.3.2 Prediction()

```
Result BayesianFiltering::KalmanFilter::Prediction (
    Eigen::VectorXd & U,
    StateSpaceLinearModel & linear_model ) [inline], [override], [virtual]
```

Prediction procedure, which will update (in place) the internal member variables $\hat{X}$ and $\hat{P}$ based on input vector U and model `linear_model`. In case of time varying systems, `linear_model` class members should be updated before calling this procedure.

Parameters

in	U	input vector of the linear system process model
in	<code>linear_model</code>	StateSpaceLinearModel object for which the filter will run

Implements [BayesianFiltering::LinearBayesianFilter](#).

7.4.3.3 Reset()

```
void BayesianFiltering::KalmanFilter::Reset (
    StateSpaceLinearModel & linear_model ) [inline], [override], [virtual]
```

Reset filter, first procedure before prediction/correction.

Parameters

in	<code>linear_model</code>	StateSpaceLinearModel object for which the filter will run
----	---------------------------	--

Implements [BayesianFiltering::LinearBayesianFilter](#).

The documentation for this class was generated from the following file:

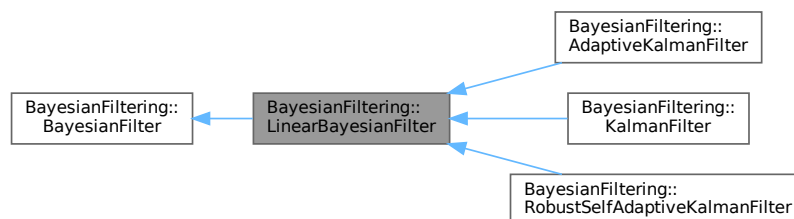
- [bayesianfiltering.hpp](#)

7.5 BayesianFiltering::LinearBayesianFilter Class Reference

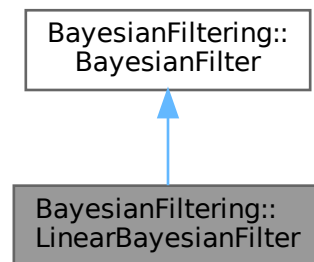
Base class of Bayesian filters for state space linear models. See derived classes.

```
#include <bayesianfiltering.hpp>
```

Inheritance diagram for BayesianFiltering::LinearBayesianFilter:



Collaboration diagram for BayesianFiltering::LinearBayesianFilter:



Public Member Functions

- virtual [~LinearBayesianFilter](#) ()
- virtual [Result Prediction](#) (Eigen::VectorXd &U, [StateSpaceLinearModel](#) &linear_model)=0
- virtual [Result Correction](#) (Eigen::VectorXd &Y, [StateSpaceLinearModel](#) &linear_model)=0
- virtual void [Reset](#) ([StateSpaceLinearModel](#) &linear_model)=0

Public Member Functions inherited from [BayesianFiltering::BayesianFilter](#)

- [BayesianFilter](#) ()
- virtual [~BayesianFilter](#) ()

Protected Member Functions

- [Result CheckModel](#) ([StateSpaceLinearModel](#) &linear_model)

Protected Member Functions inherited from [BayesianFiltering::BayesianFilter](#)

- void [Reset](#) (Eigen::VectorXd &X0, Eigen::MatrixXd &P0, int Nx, int Ny)
Reset filter, first procedure before prediction/correction.

Additional Inherited Members

Public Attributes inherited from [BayesianFiltering::BayesianFilter](#)

- Eigen::VectorXd [X](#)
State vector predictor $\hat{x}(k|k-1)$ or estimate $\hat{x}(k|k)$, depending on which stage the filter is.
- Eigen::MatrixXd [P](#)
State vector error covariance matrix predictor $P(k|k-1)$ or $P(k|k)$, depending on which stage the filter is.
- Eigen::VectorXd [Y_predicted](#)
Predicted measurement $\hat{y}(k|k-1)$.

- Eigen::VectorXd [V_predicted](#)
Predicted innovation $y(k) - y(k|k-1)$.
- Eigen::MatrixXd [S_predicted](#)
Predicted innovation covariance matrix.
- double [d2_predicted](#)
Predicted innovation Mahalanobis distance, allowing to verify a filter as optimistic or pessimistic, and divergence.
- Eigen::VectorXd [Y_corrected](#)
Corrected measurement $y(k|k)$.
- Eigen::VectorXd [V_corrected](#)
Corrected innovation $y(k) - y(k|k)$.
- Eigen::MatrixXd [S_corrected](#)
Corrected innovation covariance matrix.
- double [d2_corrected](#)
Corrected innovation Mahalanobis distance, allowing to verify a filter as optimistic or pessimistic, and divergence.

7.5.1 Detailed Description

Base class of Bayesian filters for state space linear models. See derived classes.

7.5.2 Constructor & Destructor Documentation

7.5.2.1 ~LinearBayesianFilter()

```
virtual BayesianFiltering::LinearBayesianFilter::~~LinearBayesianFilter ( ) [inline], [virtual]
```

7.5.3 Member Function Documentation

7.5.3.1 CheckModel()

```
Result BayesianFiltering::LinearBayesianFilter::CheckModel (
    StateSpaceLinearModel & linear_model ) [inline], [protected]
```

7.5.3.2 Correction()

```
virtual Result BayesianFiltering::LinearBayesianFilter::Correction (
    Eigen::VectorXd & Y,
    StateSpaceLinearModel & linear_model ) [pure virtual]
```

Implemented in [BayesianFiltering::KalmanFilter](#), [BayesianFiltering::AdaptiveKalmanFilter](#), and [BayesianFiltering::RobustSelfAdaptiveKalmanFilter](#).

7.5.3.3 Prediction()

```
virtual Result BayesianFiltering::LinearBayesianFilter::Prediction (
    Eigen::VectorXd & U,
    StateSpaceLinearModel & linear_model ) [pure virtual]
```

Implemented in [BayesianFiltering::KalmanFilter](#), [BayesianFiltering::AdaptiveKalmanFilter](#), and [BayesianFiltering::RobustSelfAdaptiveKalmanFilter](#).

7.5.3.4 Reset()

```
virtual void BayesianFiltering::LinearBayesianFilter::Reset (
    StateSpaceLinearModel & linear_model ) [pure virtual]
```

Implemented in [BayesianFiltering::KalmanFilter](#), [BayesianFiltering::AdaptiveKalmanFilter](#), and [BayesianFiltering::RobustSelfAdaptiveFilter](#).

The documentation for this class was generated from the following file:

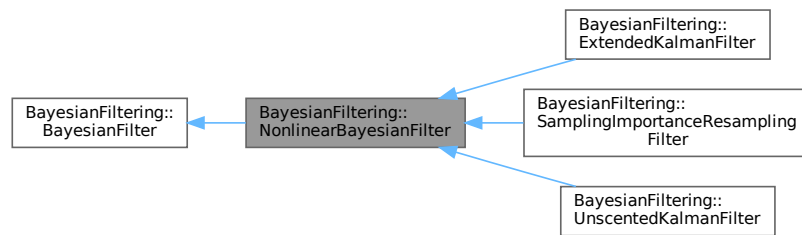
- [bayesianfiltering.hpp](#)

7.6 BayesianFiltering::NonlinearBayesianFilter Class Reference

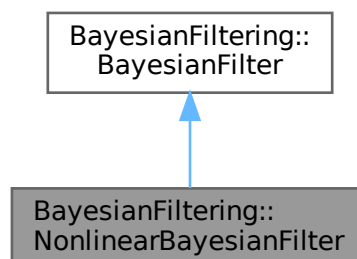
Base class of Bayesian filters for state space nonlinear models. See derived classes.

```
#include <bayesianfiltering.hpp>
```

Inheritance diagram for BayesianFiltering::NonlinearBayesianFilter:



Collaboration diagram for BayesianFiltering::NonlinearBayesianFilter:



Public Member Functions

- virtual [~NonlinearBayesianFilter](#) ()
- virtual [Result Prediction](#) (Eigen::VectorXd &U, double t, [StateSpaceNonlinearModelBase](#) &nonlinear_model)=0
- virtual [Result Correction](#) (Eigen::VectorXd &Y, double t, [StateSpaceNonlinearModelBase](#) &nonlinear_model)=0
- virtual void [Reset](#) ([StateSpaceNonlinearModelBase](#) &nonlinear_model)=0

Public Member Functions inherited from [BayesianFiltering::BayesianFilter](#)

- [BayesianFilter](#) ()
- virtual [~BayesianFilter](#) ()

Additional Inherited Members

Public Attributes inherited from [BayesianFiltering::BayesianFilter](#)

- Eigen::VectorXd [X](#)
State vector predictor $x(k|k-1)$ or estimate $x(k|k)$, depending on which stage the filter is.
- Eigen::MatrixXd [P](#)
State vector error covariance matrix predictor $P(k|k-1)$ or $P(k|k)$, depending on which stage the filter is.
- Eigen::VectorXd [Y_predicted](#)
Predicted measurement $y(k|k-1)$.
- Eigen::VectorXd [V_predicted](#)
Predicted innovation $y(k) - y(k|k-1)$.
- Eigen::MatrixXd [S_predicted](#)
Predicted innovation covariance matrix.
- double [d2_predicted](#)
Predicted innovation Mahalanobis distance, allowing to verify a filter as optimistic ou pessimistic, and divergence.
- Eigen::VectorXd [Y_corrected](#)
Corrected measurement $y(k|k)$.
- Eigen::VectorXd [V_corrected](#)
Corrected innovation $y(k) - y(k|k)$.
- Eigen::MatrixXd [S_corrected](#)
Corrected innovation covariance matrix.
- double [d2_corrected](#)
Corrected innovation Mahalanobis distance, allowing to verify a filter as optimistic ou pessimistic, and divergence.

Protected Member Functions inherited from [BayesianFiltering::BayesianFilter](#)

- void [Reset](#) (Eigen::VectorXd &X0, Eigen::MatrixXd &P0, int Nx, int Ny)
Reset filter, first procedure before prediction/correction.

7.6.1 Detailed Description

Base class of Bayesian filters for state space nonlinear models. See derived classes.

7.6.2 Constructor & Destructor Documentation

7.6.2.1 ~NonlinearBayesianFilter()

```
virtual BayesianFiltering::NonlinearBayesianFilter::~~NonlinearBayesianFilter ( ) [inline],  
[virtual]
```

7.6.3 Member Function Documentation

7.6.3.1 Correction()

```
virtual Result BayesianFiltering::NonlinearBayesianFilter::Correction (   
    Eigen::VectorXd & Y,  
    double t,  
    StateSpaceNonlinearModelBase & nonlinear_model ) [pure virtual]
```

Implemented in [BayesianFiltering::ExtendedKalmanFilter](#), [BayesianFiltering::UnscentedKalmanFilter](#), and [BayesianFiltering::SamplingImportanceResamplingFilter](#).

7.6.3.2 Prediction()

```
virtual Result BayesianFiltering::NonlinearBayesianFilter::Prediction (   
    Eigen::VectorXd & U,  
    double t,  
    StateSpaceNonlinearModelBase & nonlinear_model ) [pure virtual]
```

Implemented in [BayesianFiltering::ExtendedKalmanFilter](#), [BayesianFiltering::UnscentedKalmanFilter](#), and [BayesianFiltering::SamplingImportanceResamplingFilter](#).

7.6.3.3 Reset()

```
virtual void BayesianFiltering::NonlinearBayesianFilter::Reset (   
    StateSpaceNonlinearModelBase & nonlinear_model ) [pure virtual]
```

Implemented in [BayesianFiltering::ExtendedKalmanFilter](#), [BayesianFiltering::UnscentedKalmanFilter](#), and [BayesianFiltering::SamplingImportanceResamplingFilter](#).

The documentation for this class was generated from the following file:

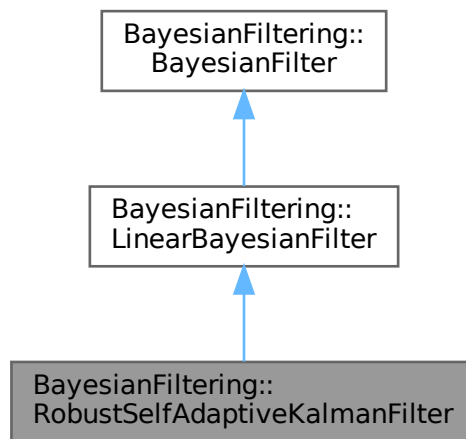
- [bayesianfiltering.hpp](#)

7.7 BayesianFiltering::RobustSelfAdaptiveKalmanFilter Class Reference

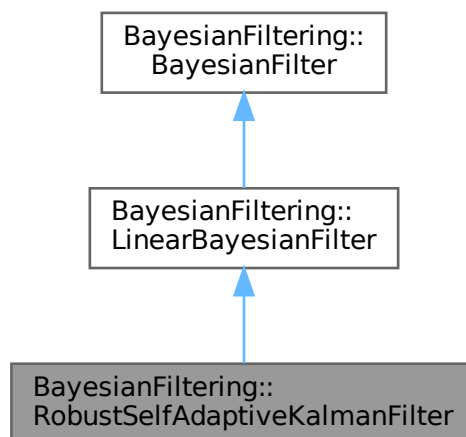
Robust Self Adaptive Kalman filter, with Q and R matrices changing when every measurement comes using covariance matching strategy.

```
#include <bayesianfiltering.hpp>
```

Inheritance diagram for BayesianFiltering::RobustSelfAdaptiveKalmanFilter:



Collaboration diagram for BayesianFiltering::RobustSelfAdaptiveKalmanFilter:



Public Member Functions

- virtual [~RobustSelfAdaptiveKalmanFilter](#) ()
- void [Reset](#) ([StateSpaceLinearModel](#) &linear_model) override
Reset filter, first procedure before prediction/correction.
- [Result Prediction](#) ([Eigen::VectorXd](#) &U, [StateSpaceLinearModel](#) &linear_model) override
Prediction procedure, which will update (in place) the internal member variables X and P based on input vector U and model `linear_model`. In case of time varying systems, `linear_model` class members should be updated before calling this procedure.
- [Result Correction](#) ([Eigen::VectorXd](#) &Y, [StateSpaceLinearModel](#) &linear_model) override
Correction procedure, which will update (in place) the internal member variables X and P based on measurement vector Y and model `linear_model`. In case of time varying systems, `linear_model` class members should be updated before calling this procedure.

Public Member Functions inherited from [BayesianFiltering::LinearBayesianFilter](#)

- virtual [~LinearBayesianFilter](#) ()

Public Member Functions inherited from [BayesianFiltering::BayesianFilter](#)

- [BayesianFilter](#) ()
- virtual [~BayesianFilter](#) ()

Additional Inherited Members

Public Attributes inherited from [BayesianFiltering::BayesianFilter](#)

- [Eigen::VectorXd](#) [X](#)
State vector predictor $x(k|k-1)$ or estimate $x(k|k)$, depending on which stage the filter is.
- [Eigen::MatrixXd](#) [P](#)
State vector error covariance matrix predictor $P(k|k-1)$ or $P(k|k)$, depending on which stage the filter is.
- [Eigen::VectorXd](#) [Y_predicted](#)
Predicted measurement $y(k|k-1)$.
- [Eigen::VectorXd](#) [V_predicted](#)
Predicted innovation $y(k) - y(k|k-1)$.
- [Eigen::MatrixXd](#) [S_predicted](#)
Predicted innovation covariance matrix.
- double [d2_predicted](#)
Predicted innovation Mahalanobis distance, allowing to verify a filter as optimistic or pessimistic, and divergence.
- [Eigen::VectorXd](#) [Y_corrected](#)
Corrected measurement $y(k|k)$.
- [Eigen::VectorXd](#) [V_corrected](#)
Corrected innovation $y(k) - y(k|k)$.
- [Eigen::MatrixXd](#) [S_corrected](#)
Corrected innovation covariance matrix.
- double [d2_corrected](#)
Corrected innovation Mahalanobis distance, allowing to verify a filter as optimistic or pessimistic, and divergence.

Protected Member Functions inherited from [BayesianFiltering::LinearBayesianFilter](#)

- [Result CheckModel](#) ([StateSpaceLinearModel](#) &linear_model)

Protected Member Functions inherited from [BayesianFiltering::BayesianFilter](#)

- void [Reset](#) (Eigen::VectorXd &X0, Eigen::MatrixXd &P0, int Nx, int Ny)
Reset filter, first procedure before prediction/correction.

7.7.1 Detailed Description

Robust Self Adaptive Kalman filter, with Q and R matrices changing when every measurement comes using covariance matching strategy.

This implementation is specific for discrete time, continuous state models.

From: Chen Y-W, Tu K-M. Robust self-adaptive Kalman filter with application in target tracking. Measurement and Control. 2022;55(9-10):935-944. doi:10.1177/00202940221083548

7.7.2 Constructor & Destructor Documentation

7.7.2.1 `~RobustSelfAdaptiveKalmanFilter()`

```
virtual BayesianFiltering::RobustSelfAdaptiveKalmanFilter::~RobustSelfAdaptiveKalmanFilter ( )
[inline], [virtual]
```

7.7.3 Member Function Documentation

7.7.3.1 `Correction()`

```
Result BayesianFiltering::RobustSelfAdaptiveKalmanFilter::Correction (
    Eigen::VectorXd & Y,
    StateSpaceLinearModel & linear_model ) [inline], [override], [virtual]
```

Correction procedure, which will update (in place) the internal member variables $\hat{X}$ and P based on measurement vector Y and model `linear_model`. In case of time varying systems, `linear_model` class members should be updated before calling this procedure.

Parameters

in	Y	measurement vector of the linear system process model
in	<code>linear_model</code>	StateSpaceLinearModel object for which the filter will run

Implements [BayesianFiltering::LinearBayesianFilter](#).

7.7.3.2 Prediction()

```
Result BayesianFiltering::RobustSelfAdaptiveKalmanFilter::Prediction (
    Eigen::VectorXd & U,
    StateSpaceLinearModel & linear_model ) [inline], [override], [virtual]
```

Prediction procedure, which will update (in place) the internal member variables $\mathbf{X}$ and $\mathbf{P}$ based on input vector $\mathbf{U}$ and model `linear_model`. In case of time varying systems, `linear_model` class members should be updated before calling this procedure.

Parameters

in	<i>U</i>	input vector of the linear system process model
in	<i>linear_model</i>	StateSpaceLinearModel object for which the filter will run

Implements [BayesianFiltering::LinearBayesianFilter](#).

7.7.3.3 Reset()

```
void BayesianFiltering::RobustSelfAdaptiveKalmanFilter::Reset (
    StateSpaceLinearModel & linear_model ) [inline], [override], [virtual]
```

Reset filter, first procedure before prediction/correction.

Parameters

in	<i>linear_model</i>	StateSpaceLinearModel object for which the filter will run
----	---------------------	--

Implements [BayesianFiltering::LinearBayesianFilter](#).

The documentation for this class was generated from the following file:

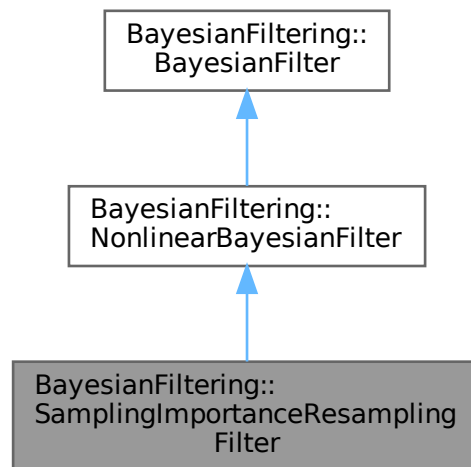
- [bayesianfiltering.hpp](#)

7.8 BayesianFiltering::SamplingImportanceResamplingFilter Class Reference

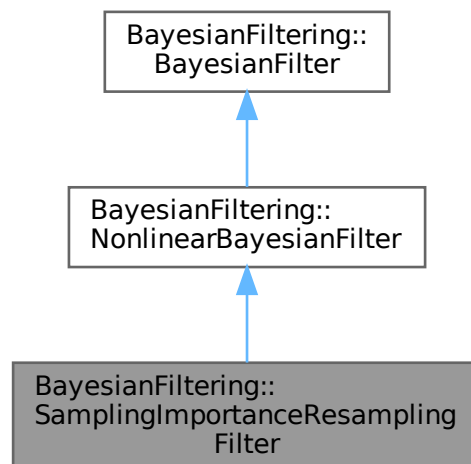
Sampling Importance Resampling (SIR) filter, particle-based estimator for stochastic nonlinear state space models.

```
#include <bayesianfiltering.hpp>
```

Inheritance diagram for BayesianFiltering::SamplingImportanceResamplingFilter:



Collaboration diagram for BayesianFiltering::SamplingImportanceResamplingFilter:



Public Types

- enum class [EstimateType](#) { [MaximumAPosteriori](#) , [ConditionalMean](#) }
- Enum of type of estimate computed by the filter at x vector.*

Public Member Functions

- virtual [~SamplingImportanceResamplingFilter](#) ()
- void [Reset](#) ([StateSpaceNonlinearModelBase](#) &nonlinear_model, long int _Nparticles, [EstimateType](#) _estimate_type)

Reset filter with user supplied filter specific parameters. Must be called before the prediction/correction cycle.
- void [Reset](#) ([StateSpaceNonlinearModelBase](#) &nonlinear_model) override

Reset filter with default filter specific parameters (Nparticles = 1000 and estimate_type = [EstimateType::Conditional](#)). Must be called before the prediction/correction cycle.
- [Result NormalizeWeights](#) ()

Normalize particle's weight such that $\sum(w_{particles}) = 1.0$.
- [Result ComputeESS](#) ()

Update ESS (Effective Sample Size) member variable.
- [Result Prediction](#) ([Eigen::VectorXd](#) &U, double t, [StateSpaceNonlinearModelBase](#) &nonlinear_model) override

Prediction procedure, which will update (in place) the internal member variables X and P based on input vector U and model `nonlinear_model`. This procedure calls methods of the derived class object `nonlinear_model` in order to update model parameters. So there is no need the user to do that.
- [Result Correction](#) ([Eigen::VectorXd](#) &Y, double t, [StateSpaceNonlinearModelBase](#) &nonlinear_model) override

Correction procedure, which will update (in place) the internal member variables X and P based on measurement vector Y and model `linear_model`. In case of time varying systems, `linear_model` class members should be updated before calling this procedure.

Public Member Functions inherited from [BayesianFiltering::NonlinearBayesianFilter](#)

- virtual [~NonlinearBayesianFilter](#) ()

Public Member Functions inherited from [BayesianFiltering::BayesianFilter](#)

- [BayesianFilter](#) ()
- virtual [~BayesianFilter](#) ()

Static Public Member Functions

- static const char * [EnumToString](#) ([EstimateType](#) x)

Generate text description Enum of type of estimate computed by the filter at X vector.

Public Attributes

- [EstimateType](#) estimate_type

Estimation type to be provided by the filter at X vector.
- long int [Nparticles](#)

Number of particles.
- [Eigen::MatrixXd](#) [Xparticles](#)

X particles, one per col
- [Eigen::MatrixXd](#) [Yparticles](#)

Y particles, one per col
- [Eigen::VectorXd](#) [Wparticles](#)

Weight for each particle, set to $1/Nparticles$ after resampling.
- double [ESS](#)

Effective Sample Size, health of particles distribution. Read after filter correction.

Public Attributes inherited from [BayesianFiltering::BayesianFilter](#)

- Eigen::VectorXd [X](#)
State vector predictor $x(k|k-1)$ or estimate $x(k|k)$, depending on which stage the filter is.
- Eigen::MatrixXd [P](#)
State vector error covariance matrix predictor $P(k|k-1)$ or $P(k|k)$, depending on which stage the filter is.
- Eigen::VectorXd [Y_predicted](#)
Predicted measurement $y(k|k-1)$.
- Eigen::VectorXd [V_predicted](#)
Predicted innovation $y(k) - y(k|k-1)$.
- Eigen::MatrixXd [S_predicted](#)
Predicted innovation covariance matrix.
- double [d2_predicted](#)
Predicted innovation Mahalanobis distance, allowing to verify a filter as optimistic ou pessimistic, and divergence.
- Eigen::VectorXd [Y_corrected](#)
Corrected measurement $y(k|k)$.
- Eigen::VectorXd [V_corrected](#)
Corrected innovation $y(k) - y(k|k)$.
- Eigen::MatrixXd [S_corrected](#)
Corrected innovation covariance matrix.
- double [d2_corrected](#)
Corrected innovation Mahalanobis distance, allowing to verify a filter as optimistic ou pessimistic, and divergence.

Additional Inherited Members

Protected Member Functions inherited from [BayesianFiltering::BayesianFilter](#)

- void [Reset](#) (Eigen::VectorXd &X0, Eigen::MatrixXd &P0, int Nx, int Ny)
Reset filter, first procedure before prediction/correction.

7.8.1 Detailed Description

Sampling Importance Resampling (SIR) filter, particle-based estimator for stochastic nonlinear state space models.

This implementation is specific for discrete time, continuous state models.

Follows algorithm described at: M. S. Arulampalam, S. Maskell, N. Gordon and T. Clapp, "A tutorial on particle filters for online nonlinear/non-Gaussian Bayesian tracking", in IEEE Transactions on Signal Processing, vol. 50, no. 2, pp. 174-188, Feb. 2002, doi: 10.1109/78.978374.

7.8.2 Member Enumeration Documentation

7.8.2.1 EstimateType

```
enum class BayesianFiltering::SamplingImportanceResamplingFilter::EstimateType [strong]
```

Enum of type of estimate computed by the filter at X vector.

Enumerator

MaximumAPosteriori	Maximum a Posteriori (MAP) estimate.
ConditionalMean	Conditional mean estimate.

7.8.3 Constructor & Destructor Documentation

7.8.3.1 ~SamplingImportanceResamplingFilter()

```
virtual BayesianFiltering::SamplingImportanceResamplingFilter::~SamplingImportanceResamplingFilter ( ) [inline], [virtual]
```

7.8.4 Member Function Documentation

7.8.4.1 ComputeESS()

```
Result BayesianFiltering::SamplingImportanceResamplingFilter::ComputeESS ( ) [inline]
```

Update ESS (Effective Sample Size) member variable.

7.8.4.2 Correction()

```
Result BayesianFiltering::SamplingImportanceResamplingFilter::Correction (
    Eigen::VectorXd & Y,
    double t,
    StateSpaceNonlinearModelBase & nonlinear_model ) [inline], [override], [virtual]
```

Correction procedure, which will update (in place) the internal member variables **X** and **P** based on measurement vector **Y** and model **linear_model**. In case of time varying systems, **linear_model** class members should be updated before calling this procedure.

Parameters

in	Y	measurement vector of the linear system process model
in	t	time variable, for time varying systems
in	nonlinear_model	StateSpaceNonlinearModelBase derived object for which the filter will run. Model updating and parameter calculations implemented by the user in the StateSpaceNonlinearModelBase derived object are called from inside this method.

Implements [BayesianFiltering::NonlinearBayesianFilter](#).

7.8.4.3 EnumToString()

```
static const char * BayesianFiltering::SamplingImportanceResamplingFilter::EnumToString (
    EstimateType x ) [inline], [static]
```

Generate text description Enum of type of estimate computed by the filter at **X** vector.

7.8.4.4 NormalizeWeights()

Result BayesianFiltering::SamplingImportanceResamplingFilter::NormalizeWeights () [inline]

Normalize particle's weight such that $\sum(w_{particles}) = 1.0$.

7.8.4.5 Prediction()

Result BayesianFiltering::SamplingImportanceResamplingFilter::Prediction (
 Eigen::VectorXd & *U*,
 double *t*,
 StateSpaceNonlinearModelBase & *nonlinear_model*) [inline], [override], [virtual]

Prediction procedure, which will update (in place) the internal member variables *X* and *P* based on input vector *U* and model *nonlinear_model*. This procedure calls methods of the derived class object *nonlinear_model* in order to update model parameters. So there is no need the user to do that.

Parameters

in	<i>U</i>	input vector of the linear system process model
in	<i>t</i>	time variable, for time varying systems
in	<i>nonlinear_model</i>	StateSpaceNonlinearModelBase derived object for which the filter will run. Model updating and parameter calculations implemented by the user in the StateSpaceNonlinearModelBase derived object are called from inside this method.

Implements BayesianFiltering::NonlinearBayesianFilter.

7.8.4.6 Reset() [1/2]

void BayesianFiltering::SamplingImportanceResamplingFilter::Reset (
 StateSpaceNonlinearModelBase & *nonlinear_model*) [inline], [override], [virtual]

Reset filter with default filter specific parameters (*Nparticles* = 1000 and *estimate_type* = EstimateType::ConditionalMean). Must be called before the prediction/correction cycle.

Parameters

in	<i>nonlinear_model</i>	StateSpaceNonlinearModelBase object for which the filter will run
----	------------------------	---

Implements BayesianFiltering::NonlinearBayesianFilter.

7.8.4.7 Reset() [2/2]

void BayesianFiltering::SamplingImportanceResamplingFilter::Reset (
 StateSpaceNonlinearModelBase & *nonlinear_model*,
 long int *_Nparticles*,
 EstimateType *_estimate_type*) [inline]

Reset filter with user supplied filter specific parameters. Must be called before the prediction/correction cycle.

Parameters

in	<i>nonlinear_model</i>	StateSpaceNonlinearModelBase object for which the filter will run
in	<i>_Nparticles</i>	user provided value for Nparticles member variable
in	<i>_estimate_type</i>	user provided value for estimate_type member variable

7.8.5 Member Data Documentation

7.8.5.1 ESS

`double BayesianFiltering::SamplingImportanceResamplingFilter::ESS`

Effective Sample Size, health of particles distribution. Read after filter correction.

7.8.5.2 estimate_type

[EstimateType](#) `BayesianFiltering::SamplingImportanceResamplingFilter::estimate_type`

Estimation type to be provided by the filter at X vector.

7.8.5.3 Nparticles

`long int BayesianFiltering::SamplingImportanceResamplingFilter::Nparticles`

Number of particles.

7.8.5.4 Wparticles

`Eigen::VectorXd BayesianFiltering::SamplingImportanceResamplingFilter::Wparticles`

Weight for each particle, set to 1/Nparticles after resampling.

7.8.5.5 Xparticles

`Eigen::MatrixXd BayesianFiltering::SamplingImportanceResamplingFilter::Xparticles`

X particles, one per col

7.8.5.6 Yparticles

`Eigen::MatrixXd BayesianFiltering::SamplingImportanceResamplingFilter::Yparticles`

Y particles, one per col

The documentation for this class was generated from the following file:

- [bayesianfiltering.hpp](#)

7.9 BayesianFiltering::StateSpaceLinearModel Class Reference

Represents the parameters of a discrete time, generic linear state space model:

```
#include <bayesianfiltering.hpp>
```

Public Member Functions

- void [SetDimensions](#) (int _Nx, int _Nu, int _Ny, int _Nw, int _Nv)
Define dimensions of system variables and adjust size of model parameters accordingly.
- [Result ProcessModel](#) (Eigen::VectorXd &X, Eigen::VectorXd &U, Eigen::VectorXd &W)
Compute process model to update state vector X (in place).
- [Result MeasurementModel](#) (Eigen::VectorXd &Y, Eigen::VectorXd &X, Eigen::VectorXd &V)
Compute system output Y from measurement model.

Public Attributes

- Eigen::MatrixXd [A](#)
Process model transition matrix.
- Eigen::MatrixXd [B](#)
Process model input matrix.
- Eigen::MatrixXd [C](#)
Measurement model matrix.
- Eigen::MatrixXd [Q](#)
Process model noise covariance matrix.
- Eigen::MatrixXd [R](#)
Measurement model noise covariance matrix.
- Eigen::VectorXd [X0](#)
Initial state vector distribution mean vector parameter.
- Eigen::MatrixXd [P0](#)
Initial state vector distribution covariance matrix parameter.
- int [Nx](#)
Number of state variables in X vector.
- int [Nu](#)
Number of inputs in U vector.
- int [Ny](#)
Number of measurements in Y vector.
- int [Nw](#)
Number of process noise in W vector.
- int [Nv](#)
Number of measurement noise in V vector.
- std::vector< bool > [X_isangle](#)
Indicates which entries of X vector are angles in rad, for correct computation of correction equation.
- std::vector< bool > [Y_isangle](#)
Indicates which entries of Y vector are angles in rad, for correct computation of innovation vector.

7.9.1 Detailed Description

Represents the parameters of a discrete time, generic linear state space model:

$$\begin{aligned} \mathbf{x}(k) &= \mathbf{A}\mathbf{x}(k-1) + \mathbf{B}\mathbf{u}(k) + \mathbf{w}(k) \quad \{\text{Process equation}\} \\ \mathbf{y}(k) &= \mathbf{C}\mathbf{x}(k) + \mathbf{v}(k) \quad \{\text{Measurement equation}\} \end{aligned}$$

with $\mathbf{x}(k)$ being state variables vector, $\mathbf{u}(k)$ measurement vector and $\mathbf{y}(k)$ measurement vector. Also, $\mathbf{w}(k) \sim N(0, Q)$ and $\mathbf{v}(k) \sim N(0, R)$ are respectively process and measurement noises.

This class contains only model parameters and evaluations of process and measurement equations. In case $\mathbf{w}(k)$ and/or $\mathbf{v}(k)$ are dependent of other noise sources, Q and R should consider the correct propagation from noise sources covariances.

Initial state is a random variable following $\mathbf{x}(0) \sim N(\mathbf{X0}, \mathbf{P0})$

Also $\dim(\mathbf{x}) = N_x$, $\dim(\mathbf{u}) = N_u$, $\dim(\mathbf{y}) = N_y$, $\dim(\mathbf{w}) = N_w$, $\dim(\mathbf{v}) = N_v$. These dimensions should be first stablished by calling `SetDimensions()`, which make resizing of system parameters. After that, since parameters are public, they can be set at any time.

Angular measurements in radians need special treatment when computing innovation vector in filters using `atan2` function. In order to do this, `Y_isangle` is used to indicate which components of $\mathbf{Y}$ are angles, which should be set as `true` by the user when initializing the model.

For the same reason, every change in state vector containing angles in some entries should me donne properly. Thus, `X_isangle` is used to indicate which components of $\mathbf{x}$ are angles, which should be set as `true` by the user when initializing the model.

7.9.2 Member Function Documentation

7.9.2.1 MeasurementModel()

```
Result BayesianFiltering::StateSpaceLinearModel::MeasurementModel (
    Eigen::VectorXd & Y,
    Eigen::VectorXd & X,
    Eigen::VectorXd & V ) [inline]
```

Compute system output $\mathbf{Y}$ from measurement model.

Parameters

out	$\mathbf{Y}$	the computed output
in	$\mathbf{X}$	state vector with $\mathbf{x}(k)$
in	$\mathbf{V}$	current sample of measurement noise

7.9.2.2 ProcessModel()

```
Result BayesianFiltering::StateSpaceLinearModel::ProcessModel (
    Eigen::VectorXd & X,
```

```
Eigen::VectorXd & U,
Eigen::VectorXd & W ) [inline]
```

Compute process model to update state vector X (in place).

Parameters

in, out	X	state vector with $x(k-1)$ to be updated to $x(k)$
in	U	input vector
in	W	current sample of process noise

7.9.2.3 SetDimensions()

```
void BayesianFiltering::StateSpaceLinearModel::SetDimensions (
    int _Nx,
    int _Nu,
    int _Ny,
    int _Nw,
    int _Nv ) [inline]
```

Define dimensions of system variables and adjust size of model parameters accordingly.

Parameters

in	$_Nx$	Number of state variables in X vector
in	$_Nu$	Number of inputs in U vector
in	$_Ny$	Number of measurements in Y vector
in	$_Nw$	Number of process noise in W vector
in	$_Nv$	Number of measurement noise in V vector

7.9.3 Member Data Documentation

7.9.3.1 A

```
Eigen::MatrixXd BayesianFiltering::StateSpaceLinearModel::A
```

Process model transition matrix.

7.9.3.2 B

```
Eigen::MatrixXd BayesianFiltering::StateSpaceLinearModel::B
```

Process model input matrix.

7.9.3.3 C

```
Eigen::MatrixXd BayesianFiltering::StateSpaceLinearModel::C
```

Measurement model matrix.

7.9.3.4 Nu

```
int BayesianFiltering::StateSpaceLinearModel::Nu
```

Number of inputs in U vector.

7.9.3.5 Nv

```
int BayesianFiltering::StateSpaceLinearModel::Nv
```

Number of measurement noise in V vector.

7.9.3.6 Nw

```
int BayesianFiltering::StateSpaceLinearModel::Nw
```

Number of process noise in W vector.

7.9.3.7 Nx

```
int BayesianFiltering::StateSpaceLinearModel::Nx
```

Number of state variables in X vector.

7.9.3.8 Ny

```
int BayesianFiltering::StateSpaceLinearModel::Ny
```

Number of measurements in Y vector.

7.9.3.9 P0

```
Eigen::MatrixXd BayesianFiltering::StateSpaceLinearModel::P0
```

Initial state vector distribution covariance matrix parameter.

7.9.3.10 Q

```
Eigen::MatrixXd BayesianFiltering::StateSpaceLinearModel::Q
```

Process model noise covariance matrix.

7.9.3.11 R

```
Eigen::MatrixXd BayesianFiltering::StateSpaceLinearModel::R
```

Measurement model noise covariance matrix.

7.9.3.12 X0

`Eigen::VectorXd BayesianFiltering::StateSpaceLinearModel::X0`

Initial state vector distribution mean vector parameter.

7.9.3.13 X_isangle

`std::vector<bool> BayesianFiltering::StateSpaceLinearModel::X_isangle`

Indicates which entries of X vector are angles in rad, for correct computation of correction equation.

7.9.3.14 Y_isangle

`std::vector<bool> BayesianFiltering::StateSpaceLinearModel::Y_isangle`

Indicates which entries of Y vector are angles in rad, for correct computation of innovation vector.

The documentation for this class was generated from the following file:

- [bayesianfiltering.hpp](#)

7.10 BayesianFiltering::StateSpaceNonlinearModelBase Class Reference

Represents the parameters of a discrete time, generic nonlinear state space model:

```
#include <bayesianfiltering.hpp>
```

Public Member Functions

- virtual `~StateSpaceNonlinearModelBase()`=default
- void `SetDimensions` (int _Nx, int _Nu, int _Ny, int _Nw, int _Nv)
Define dimensions of system variables and adjust size of model parameters accordingly.
- virtual `Result ProcessModel` (Eigen::VectorXd &X, Eigen::VectorXd &U, double t, Eigen::VectorXd &W)=0
Compute process model to update state vector x (in place). Virtual function to be defined in derived class.
- virtual `Result MeasurementModel` (Eigen::VectorXd &Y, Eigen::VectorXd &X, double t, Eigen::VectorXd &V)=0
Compute system output y from measurement model. Virtual function to be defined in derived class.
- virtual `Result UpdatedfdX` (Eigen::VectorXd &X, Eigen::VectorXd &U, double t, Eigen::VectorXd &W)=0
Update $dfdX$ internal class member, the process model derivative with respect to $x(k-1)$. Necessary only in filters using system matrix derivatives, where this function is called from inside the filter class. Virtual function to be defined in derived class.
- virtual `Result UpdatedfdW` (Eigen::VectorXd &X, Eigen::VectorXd &U, double t, Eigen::VectorXd &W)=0
Update $dfdW$ internal class member, the process model derivative with respect to $w(k)$. Necessary only in filters using system matrix derivatives, where this function is called from inside the filter class. Virtual function to be defined in derived class.
- virtual `Result UpdatedhdX` (Eigen::VectorXd &X, double t, Eigen::VectorXd &V)=0
Update $dhdX$ internal class member, the measurement model derivative with respect to $x(k)$. Necessary only in filters using system matrix derivatives, where this function is called from inside the filter class. Virtual function to be defined in derived class.
- virtual `Result UpdatedhdV` (Eigen::VectorXd &X, double t, Eigen::VectorXd &V)=0
Update $dhdV$ internal class member, the measurement model derivative with respect to $v(k)$. Necessary only in filters using system matrix derivatives, where this function is called from inside the filter class. Virtual function to be defined in derived class.

Public Attributes

- Eigen::MatrixXd [dfdX](#)
Partial derivative of $f(X,U,W)$ with respect to X .
- Eigen::MatrixXd [dfdW](#)
Partial derivative of $f(X,U,W)$ with respect to W .
- Eigen::MatrixXd [dhdX](#)
Partial derivative of $h(X,V)$ with respect to X .
- Eigen::MatrixXd [dhdV](#)
Partial derivative of $h(X,V)$ with respect to V .
- Eigen::MatrixXd [Q](#)
Process model noise covariance matrix.
- Eigen::MatrixXd [R](#)
Measurement model noise covariance matrix.
- Eigen::VectorXd [X0](#)
Initial state vector distribution mean vector parameter.
- Eigen::MatrixXd [P0](#)
Initial state vector distribution covariance matrix parameter.
- int [Nx](#)
Number of state variables in X vector.
- int [Nu](#)
Number of inputs in U vector.
- int [Ny](#)
Number of measurements in Y vector.
- int [Nw](#)
Number of process noise in W vector.
- int [Nv](#)
Number of measurement noise in V vector.
- std::vector< bool > [X_isangle](#)
Indicates which entries of X vector are angles in rad, for correct computation of correction equation.
- std::vector< bool > [Y_isangle](#)
Indicates which entries of Y vector are angles in rad, for correct computation of innovation vector.

7.10.1 Detailed Description

Represents the parameters of a discrete time, generic nonlinear state space model:

$$\begin{aligned} \mathbf{x}(k) &= \mathbf{f}(\mathbf{x}(k-1), \mathbf{u}(k), \mathbf{w}(k)) \quad \{\text{Process equation}\} \\ \mathbf{y}(k) &= \mathbf{h}(\mathbf{x}(k), \mathbf{v}(k)) \quad \{\text{Measurement equation}\} \end{aligned}$$

with $\mathbf{x}(k)$ being state variables vector, $\mathbf{u}(k)$ measurement vector and $\mathbf{y}(k)$ measurement vector. Also, $\mathbf{w}(k) \sim N(0, Q)$ and $\mathbf{v}(k) \sim N(0, R)$ are respectively process and measurement noises.

This class contains only model parameters. It is a base virtual class, with computation of process and measurement equations as virtual member functions. Its definition should be done in a derived class at user side. Partial derivatives of process and measurement models should also be defined at a derived class. Some filters use such derivatives.

Initial state is a random variable following $\mathbf{x}(0) \sim N(\mathbf{X0}, \mathbf{P0})$

Also $\dim(x) = N_x$, $\dim(u) = N_u$, $\dim(y) = N_y$, $\dim(w) = N_w$, $\dim(v) = N_v$. These dimensions should be first established by calling `SetDimensions()`, which make resizing of system parameters. After that, since parameters are public, they can be set at any time.

Angular measurements in radians need special treatment when computing innovation vector in filters using `atan2` function. In order to do this, `Y_isangle` is used to indicate which components of `Y` are angles, which should be set as `true` by the user when initializing the model.

For the same reason, every change in state vector containing angles in some entries should be done properly. Also, some filters compute covariance matrix from samples, requiring special treatment. Thus, `X_isangle` is used to indicate which components of `x` are angles, which should be set as `true` by the user when initializing the model.

7.10.2 Constructor & Destructor Documentation

7.10.2.1 ~StateSpaceNonlinearModelBase()

```
virtual BayesianFiltering::StateSpaceNonlinearModelBase::~StateSpaceNonlinearModelBase ( )
[virtual], [default]
```

7.10.3 Member Function Documentation

7.10.3.1 MeasurementModel()

```
virtual Result BayesianFiltering::StateSpaceNonlinearModelBase::MeasurementModel (
    Eigen::VectorXd & Y,
    Eigen::VectorXd & X,
    double t,
    Eigen::VectorXd & V ) [pure virtual]
```

Compute system output `Y` from measurement model. Virtual function to be defined in derived class.

Parameters

out	<code>Y</code>	the computed output
in	<code>X</code>	state vector with $x(k)$
in	<code>t</code>	time instant scalar
in	<code>V</code>	current sample of measurement noise

7.10.3.2 ProcessModel()

```
virtual Result BayesianFiltering::StateSpaceNonlinearModelBase::ProcessModel (
    Eigen::VectorXd & X,
    Eigen::VectorXd & U,
    double t,
    Eigen::VectorXd & W ) [pure virtual]
```

Compute process model to update state vector `X` (in place). Virtual function to be defined in derived class.

Parameters

in, out	X	state vector with $x(k-1)$ to be updated to $x(k)$
in	U	input vector
in	t	time instant scalar
in	W	current sample of process noise

7.10.3.3 SetDimensions()

```
void BayesianFiltering::StateSpaceNonlinearModelBase::SetDimensions (
    int _Nx,
    int _Nu,
    int _Ny,
    int _Nw,
    int _Nv ) [inline]
```

Define dimensions of system variables and adjust size of model parameters accordingly.

Parameters

in	$_Nx$	Number of state variables in X vector
in	$_Nu$	Number of inputs in U vector
in	$_Ny$	Number of measurements in Y vector
in	$_Nw$	Number of process noise in W vector
in	$_Nv$	Number of measurement noise in V vector

7.10.3.4 UpdatedfdW()

```
virtual Result BayesianFiltering::StateSpaceNonlinearModelBase::UpdatedfdW (
    Eigen::VectorXd & X,
    Eigen::VectorXd & U,
    double t,
    Eigen::VectorXd & W ) [pure virtual]
```

Update dfdW internal class member, the process model derivative with respect to $w(k)$. Necessary only in filters using system matrix derivatives, where this function is called from inside the filter class. Virtual function to be defined in derived class.

Parameters

in	X	state vector containing $x(k-1)$
in	U	input vector
in	t	time instant scalar
in	W	current sample of process noise

7.10.3.5 UpdatedfdX()

```
virtual Result BayesianFiltering::StateSpaceNonlinearModelBase::UpdatedfdX (
```

```

Eigen::VectorXd & X,
Eigen::VectorXd & U,
double t,
Eigen::VectorXd & W ) [pure virtual]

```

Update dfdX internal class member, the process model derivative with respect to $\mathbf{x}(k-1)$. Necessary only in filters using system matrix derivatives, where this function is called from inside the filter class. Virtual function to be defined in derived class.

Parameters

in	X	state vector containing $\mathbf{x}(k-1)$
in	U	input vector
in	t	time instant scalar
in	W	current sample of process noise

7.10.3.6 UpdatedhdV()

```

virtual Result BayesianFiltering::StateSpaceNonlinearModelBase::UpdatedhdV (
    Eigen::VectorXd & X,
    double t,
    Eigen::VectorXd & V ) [pure virtual]

```

Update dhdV internal class member, the measurement model derivative with respect to $\mathbf{v}(k)$. Necessary only in filters using system matrix derivatives, where this function is called from inside the filter class. Virtual function to be defined in derived class.

Parameters

in	X	state vector containing $\mathbf{x}(k-1)$
in	t	time instant scalar
in	V	current sample of measurement noise

7.10.3.7 UpdatedhdX()

```

virtual Result BayesianFiltering::StateSpaceNonlinearModelBase::UpdatedhdX (
    Eigen::VectorXd & X,
    double t,
    Eigen::VectorXd & V ) [pure virtual]

```

Update dhdX internal class member, the measurement model derivative with respect to $\mathbf{x}(k)$. Necessary only in filters using system matrix derivatives, where this function is called from inside the filter class. Virtual function to be defined in derived class.

Parameters

in	X	state vector containing $\mathbf{x}(k-1)$
in	t	time instant scalar
in	V	current sample of measurement noise

7.10.4 Member Data Documentation

7.10.4.1 dfdW

`Eigen::MatrixXd BayesianFiltering::StateSpaceNonlinearModelBase::dfdW`

Partial derivative of $f(X,U,W)$ with respect to W .

7.10.4.2 dfdX

`Eigen::MatrixXd BayesianFiltering::StateSpaceNonlinearModelBase::dfdX`

Partial derivative of $f(X,U,W)$ with respect to X .

7.10.4.3 dhvV

`Eigen::MatrixXd BayesianFiltering::StateSpaceNonlinearModelBase::dhvV`

Partial derivative of $h(X,V)$ with respect to V .

7.10.4.4 dhvX

`Eigen::MatrixXd BayesianFiltering::StateSpaceNonlinearModelBase::dhvX`

Partial derivative of $h(X,V)$ with respect to X .

7.10.4.5 Nu

`int BayesianFiltering::StateSpaceNonlinearModelBase::Nu`

Number of inputs in U vector.

7.10.4.6 Nv

`int BayesianFiltering::StateSpaceNonlinearModelBase::Nv`

Number of measurement noise in V vector.

7.10.4.7 Nw

`int BayesianFiltering::StateSpaceNonlinearModelBase::Nw`

Number of process noise in W vector.

7.10.4.8 Nx

```
int BayesianFiltering::StateSpaceNonlinearModelBase::Nx
```

Number of state variables in X vector.

7.10.4.9 Ny

```
int BayesianFiltering::StateSpaceNonlinearModelBase::Ny
```

Number of measurements in Y vector.

7.10.4.10 P0

```
Eigen::MatrixXd BayesianFiltering::StateSpaceNonlinearModelBase::P0
```

Initial state vector distribution covariance matrix parameter.

7.10.4.11 Q

```
Eigen::MatrixXd BayesianFiltering::StateSpaceNonlinearModelBase::Q
```

Process model noise covariance matrix.

7.10.4.12 R

```
Eigen::MatrixXd BayesianFiltering::StateSpaceNonlinearModelBase::R
```

Measurement model noise covariance matrix.

7.10.4.13 X0

```
Eigen::VectorXd BayesianFiltering::StateSpaceNonlinearModelBase::X0
```

Initial state vector distribution mean vector parameter.

7.10.4.14 X_isangle

```
std::vector<bool> BayesianFiltering::StateSpaceNonlinearModelBase::X_isangle
```

Indicates which entries of X vector are angles in rad, for correct computation of correction equation.

7.10.4.15 Y_isangle

```
std::vector<bool> BayesianFiltering::StateSpaceNonlinearModelBase::Y_isangle
```

Indicates which entries of Y vector are angles in rad, for correct computation of innovation vector.

The documentation for this class was generated from the following file:

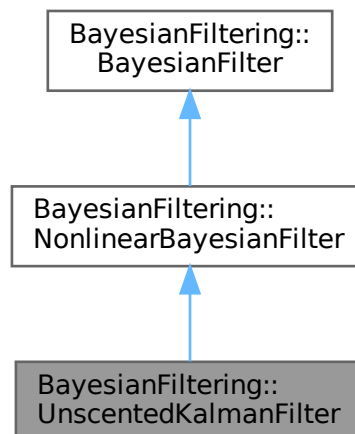
- [bayesianfiltering.hpp](#)

7.11 BayesianFiltering::UnscentedKalmanFilter Class Reference

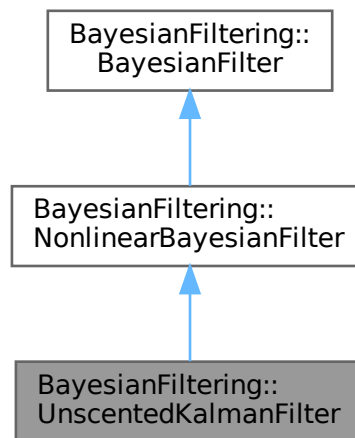
Unscented Kalman filter, the Unscented Transform (UT)-based estimator for stochastic nonlinear state space models.

```
#include <bayesianfiltering.hpp>
```

Inheritance diagram for BayesianFiltering::UnscentedKalmanFilter:



Collaboration diagram for BayesianFiltering::UnscentedKalmanFilter:



Public Member Functions

- virtual `~UnscentedKalmanFilter()`
- void `Reset(StateSpaceNonlinearModelBase &nonlinear_model)` override
Reset filter, first procedure before prediction/correction.
- `Result Prediction(Eigen::VectorXd &U, double t, StateSpaceNonlinearModelBase &nonlinear_model)` override
Prediction procedure, which will update (in place) the internal member variables X and P based on input vector U and model `nonlinear_model`. This procedure calls methods of the derived class object `nonlinear_model` in order to update model parameters. So there is no need the user to do that.
- `Result Correction(Eigen::VectorXd &Y, double t, StateSpaceNonlinearModelBase &nonlinear_model)` override
Correction procedure, which will update (in place) the internal member variables X and P based on measurement vector Y and model `linear_model`. In case of time varying systems, `linear_model` class members should be updated before calling this procedure.

Public Member Functions inherited from BayesianFiltering::NonlinearBayesianFilter

- virtual `~NonlinearBayesianFilter()`

Public Member Functions inherited from BayesianFiltering::BayesianFilter

- `BayesianFilter()`
- virtual `~BayesianFilter()`

Additional Inherited Members

Public Attributes inherited from BayesianFiltering::BayesianFilter

- Eigen::VectorXd [X](#)
State vector predictor $\hat{x}(k|k-1)$ or estimate $\hat{x}(k|k)$, depending on which stage the filter is.
- Eigen::MatrixXd [P](#)
State vector error covariance matrix predictor $P(k|k-1)$ or $P(k|k)$, depending on which stage the filter is.
- Eigen::VectorXd [Y_predicted](#)
Predicted measurement $\hat{y}(k|k-1)$.
- Eigen::VectorXd [V_predicted](#)
Predicted innovation $y(k) - \hat{y}(k|k-1)$.
- Eigen::MatrixXd [S_predicted](#)
Predicted innovation covariance matrix.
- double [d2_predicted](#)
Predicted innovation Mahalanobis distance, allowing to verify a filter as optimistic or pessimistic, and divergence.
- Eigen::VectorXd [Y_corrected](#)
Corrected measurement $\hat{y}(k|k)$.
- Eigen::VectorXd [V_corrected](#)
Corrected innovation $y(k) - \hat{y}(k|k)$.
- Eigen::MatrixXd [S_corrected](#)
Corrected innovation covariance matrix.
- double [d2_corrected](#)
Corrected innovation Mahalanobis distance, allowing to verify a filter as optimistic or pessimistic, and divergence.

Protected Member Functions inherited from BayesianFiltering::BayesianFilter

- void [Reset](#) (Eigen::VectorXd &X0, Eigen::MatrixXd &P0, int Nx, int Ny)
Reset filter, first procedure before prediction/correction.

7.11.1 Detailed Description

Unscented Kalman filter, the Unscented Transform (UT)-based estimator for stochastic nonlinear state space models.

This implementation is specific for discrete time, continuous state models.

7.11.2 Constructor & Destructor Documentation

7.11.2.1 ~UnscentedKalmanFilter()

```
virtual BayesianFiltering::UnscentedKalmanFilter::~~UnscentedKalmanFilter ( ) [inline], [virtual]
```

7.11.3 Member Function Documentation

7.11.3.1 Correction()

Result BayesianFiltering::UnscentedKalmanFilter::Correction (
 Eigen::VectorXd & Y,
 double t,
 StateSpaceNonlinearModelBase & nonlinear_model) [inline], [override], [virtual]

Correction procedure, which will update (in place) the internal member variables X and P based on measurement vector Y and model linear_model. In case of time varying systems, linear_model class members should be updated before calling this procedure.

Parameters

in	Y	measurement vector of the linear system process model
in	t	time variable, for time varying systems
in	<i>nonlinear_model</i>	StateSpaceNonlinearModelBase derived object for which the filter will run. Model updating and parameter calculations implemented by the user in the StateSpaceNonlinearModelBase derived object are called from inside this method.

Implements [BayesianFiltering::NonlinearBayesianFilter](#).

7.11.3.2 Prediction()

```
Result BayesianFiltering::UnscentedKalmanFilter::Prediction (
    Eigen::VectorXd & U,
    double t,
    StateSpaceNonlinearModelBase & nonlinear_model ) [inline], [override], [virtual]
```

Prediction procedure, which will update (in place) the internal member variables X and P based on input vector U and model *nonlinear_model*. This procedure calls methods of the derived class object *nonlinear_model* in order to update model parameters. So there is no need the user to do that.

Parameters

in	U	input vector of the linear system process model
in	t	time variable, for time varying systems
in	<i>nonlinear_model</i>	StateSpaceNonlinearModelBase derived object for which the filter will run. Model updating and parameter calculations implemented by the user in the StateSpaceNonlinearModelBase derived object are called from inside this method.

Implements [BayesianFiltering::NonlinearBayesianFilter](#).

7.11.3.3 Reset()

```
void BayesianFiltering::UnscentedKalmanFilter::Reset (
    StateSpaceNonlinearModelBase & nonlinear_model ) [inline], [override], [virtual]
```

Reset filter, first procedure before prediction/correction.

Parameters

in	<i>nonlinear_model</i>	StateSpaceNonlinearModelBase derived object for which the filter will run
----	------------------------	---

Implements [BayesianFiltering::NonlinearBayesianFilter](#).

The documentation for this class was generated from the following file:

- [bayesianfiltering.hpp](#)

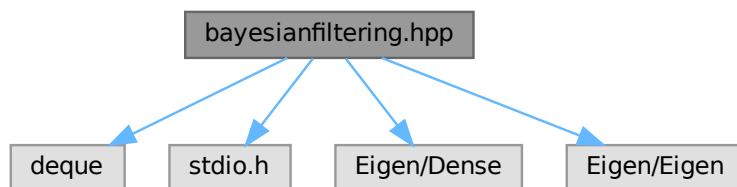
Chapter 8

File Documentation

8.1 bayesianfiltering.hpp File Reference

```
#include <deque>
#include <stdio.h>
#include <Eigen/Dense>
#include <Eigen/Eigen>
```

Include dependency graph for bayesianfiltering.hpp:



Classes

- class [BayesianFiltering::StateSpaceLinearModel](#)
Represents the parameters of a discrete time, generic linear state space model:
- class [BayesianFiltering::StateSpaceNonlinearModelBase](#)
Represents the parameters of a discrete time, generic nonlinear state space model:
- class [BayesianFiltering::BayesianFilter](#)
Base class of minimum variables of a Bayesian stochastic filter. All derived classes present filters for linear and nonlinear models. All filters follow the following loop.
- class [BayesianFiltering::LinearBayesianFilter](#)
Base class of Bayesian filters for state space linear models. See derived classes.
- class [BayesianFiltering::KalmanFilter](#)
Kalman filter, the best optimal unbiased estimator for stochastic linear state space models.

- class [BayesianFiltering::AdaptiveKalmanFilter](#)
Adaptive Kalman filter, with Q and R matrices changing when every measurement comes according two user supplied learning parameters.
- class [BayesianFiltering::RobustSelfAdaptiveKalmanFilter](#)
Robust Self Adaptive Kalman filter, with Q and R matrices changing when every measurement comes using covariance matching strategy.
- class [BayesianFiltering::NonlinearBayesianFilter](#)
Base class of Bayesian filters for state space nonlinear models. See derived classes.
- class [BayesianFiltering::ExtendedKalmanFilter](#)
Extended Kalman filter, the first order approximation-based estimator for stochastic nonlinear state space models.
- class [BayesianFiltering::UnscentedKalmanFilter](#)
Unscented Kalman filter, the Unscented Transform (UT)-based estimator for stochastic nonlinear state space models.
- class [BayesianFiltering::SamplingImportanceResamplingFilter](#)
Sampling Importance Resampling (SIR) filter, particle-based estimator for stochastic nonlinear state space models.

Namespaces

- namespace [BayesianFiltering](#)

Macros

- `#define PRINT_LINE_DEBUG {printf("\n[%s] line %i", __FILE__, __LINE__); fflush(stdout);}`
Print filename followed by line number at command line. Used commonly for debugging allowing to identify the code responsible for errors.
- `#define PRINT_EIGEN_MATRIX(X) PrintEigenMatrixXd(X,#X);`
Print pretty matrix X with the same variable name at command line (printf). Guaranteed print all lines before leaving (fflush).
- `#define PRINT_EIGEN_VECTOR(X) PrintEigenVectorXd(X,#X);`
Print pretty vector X with the same variable name at command line (printf). Guaranteed print all lines before leaving (fflush).

Typedefs

- `typedef std::pair< bool, std::string > BayesianFiltering::Result`
Encapsulated `<bool,std::string>` pair response of a function, with .first specifying status (success of failure) and .second the reason string in case of failure.

Functions

- `void PrintEigenMatrixXd (Eigen::MatrixXd &X, const char *name)`
Print pretty matrix X with name which may me different of "X" at command line (printf). Guaranteed print all lines before leaving (fflush).
- `void PrintEigenVectorXd (Eigen::VectorXd &X, const char *name)`
Print pretty vector X with name which may me different of "X" at command line (printf). Guaranteed print all lines before leaving (fflush).
- `Result BayesianFiltering::ResultFormat (bool success, const std::string &message)`
Format `<bool,std::string>` pair response with defined status and reason string.
- `template<typename ... Args>`
`Result BayesianFiltering::ResultFormat (bool success, const std::string &format, Args ... args)`
Format `<bool,std::string>` pair response with defined status and reason string to be printf-like formatted.

- [Result BayesianFiltering::MultivariateZeroMeanGaussianSampler](#) (Eigen::VectorXd &sample, Eigen::MatrixXd &covariance_matrix)
Generate multivariate $sample \sim N(0, covariance_matrix)$.
- [Result BayesianFiltering::MultivariateGaussianSampler](#) (Eigen::VectorXd &sample, Eigen::VectorXd &mean_vector, Eigen::MatrixXd &covariance_matrix)
Generate multivariate $sample \sim N(mean_vector, covariance_matrix)$.
- double [BayesianFiltering::UnivariateGaussianSampler](#) (double mean, double variance)
Generate univariate $sample \sim N(mean, variance)$.
- double [BayesianFiltering::UnivariateUniformSampler](#) (double min_value, double max_value)
Generate univariate $sample \sim U(min_value, max_value)$.

8.1.1 Macro Definition Documentation

8.1.1.1 PRINT_EIGEN_MATRIX

```
#define PRINT_EIGEN_MATRIX(  
    X ) PrintEigenMatrixXd(X, #X);
```

Print pretty matrix X with the same variable name at command line (printf). Guaranteed print all lines before leaving (fflush).

Parameters

in	X	The EIGEN::MatrixXd variable.
----	---	-------------------------------

8.1.1.2 PRINT_EIGEN_VECTOR

```
#define PRINT_EIGEN_VECTOR(  
    X ) PrintEigenVectorXd(X, #X);
```

Print pretty vector X with the same variable name at command line (printf). Guaranteed print all lines before leaving (fflush).

Parameters

in	X	The EIGEN::VectorXd variable.
----	---	-------------------------------

8.1.1.3 PRINT_LINE_DEBUG

```
#define PRINT_LINE_DEBUG {printf("\n[%s] line %i", __FILE__, __LINE__); fflush(stdout);}
```

Print filename followed by line number at command line. Used commonly for debugging allowing to identify the code responsible for errors.

Parameters

in	X	The EIGEN::VectorXd variable.
----	---	-------------------------------

8.1.2 Function Documentation

8.1.2.1 PrintEigenMatrixXd()

```
void PrintEigenMatrixXd (
    Eigen::MatrixXd & X,
    const char * name ) [inline]
```

Print pretty matrix X with name which may be different of "X" at command line (printf). Guaranteed print all lines before leaving (fflush).

Parameters

in	<i>X</i>	The EIGEN::MatrixXd variable.
in	<i>name</i>	Name of the variable.

8.1.2.2 PrintEigenVectorXd()

```
void PrintEigenVectorXd (
    Eigen::VectorXd & X,
    const char * name ) [inline]
```

Print pretty vector X with name which may be different of "X" at command line (printf). Guaranteed print all lines before leaving (fflush).

Parameters

in	<i>X</i>	The EIGEN::VectorXd variable.
in	<i>name</i>	Name of the variable.

8.2 bayesianfiltering.hpp

[Go to the documentation of this file.](#)

```
00001
00002 /*****
00003 *****/
00004 *** File:      BayesianFiltering.hpp
00005 *** Author:    Geovany A. Borges
00006 *** Contents:  BayesianFiltering header file.
00007 *****/
00008 *****/
00009 BayesianFiltering.hpp is free software: you can redistribute it and/or modify
00010 it under the terms of the GNU Lesser General Public License (LGPL) as published by
00011 the Free Software Foundation, either version 3 of the License, or any later version.
00012
00013 BayesianFiltering.hpp is distributed in the hope that it will be useful,
00014 but WITHOUT ANY WARRANTY; without even the implied warranty of
00015 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00016 GNU Lesser General Public License for more details.
00017
00018 You should have received a copy of the GNU Lesser General Public License
00019 along with BayesianFiltering.hpp. If not, see <http://www.gnu.org/licenses/>.
00020
00021 Copyright 2020 Geovany Araujo Borges
00022 *****/
00023
00024 //*****
00025 /*** Classe BayesianFiltering
```

```

00026 //*****
00027 #include <deque>
00028 #include <stdio.h>
00029
00030 #include <Eigen/Dense>
00031 #include <Eigen/Eigen>
00032
00033 #ifndef BayesianFiltering_H
00034 #define BayesianFiltering_H
00035
00036 /*****
00037  * Auxiliary Macros:
00038  *****/
00043 #define PRINT_LINE_DEBUG {printf("\n[%s] line %i", __FILE__, __LINE__); fflush(stdout);}
00044
00050 inline void PrintEigenMatrixXd(Eigen::MatrixXd &X, const char* name)
00051 {
00052     printf("\nMatrix %s (%li x %li):", name, X.rows(), X.cols());
00053     for (long int i=0; i < X.rows(); ++i){
00054         printf("\n    ");
00055         for (long int j=0; j < X.cols(); ++j){
00056             printf("%s(%li,%li)=%.10f; ", name, i, j, X(i, j));
00057         }
00058     }
00059     fflush(stdout);
00060 }
00061
00066 #define PRINT_EIGEN_MATRIX(X) PrintEigenMatrixXd(X, #X);
00067
00073 inline void PrintEigenVectorXd(Eigen::VectorXd &X, const char* name)
00074 {
00075     printf("\nVector %s (%li x %li):", name, X.rows(), X.cols());
00076     for (long int i=0; i < X.rows(); ++i){
00077         printf("\n    ");
00078         for (long int j=0; j < X.cols(); ++j){
00079             printf("%s(%li,%li)=%.10f; ", name, i, j, X(i, j));
00080         }
00081     }
00082     fflush(stdout);
00083 }
00088 #define PRINT_EIGEN_VECTOR(X) PrintEigenVectorXd(X, #X);
00089
00090 namespace BayesianFiltering {
00091
00092 /*****
00093  * Support types and functions:
00094  *****/
00099 typedef std::pair<bool, std::string> Result;
00100
00106 inline Result ResultFormat(bool success, const std::string& message) {
00107     return std::make_pair(success, message);
00108 }
00109
00116 template<typename ... Args>
00117 Result ResultFormat(bool success, const std::string& format, Args ... args )
00118 {
00119     Result res; res.first = success;
00120     int size_s = std::snprintf( nullptr, 0, format.c_str(), args ... ) + 1; // Extra space for
00121     '\0'
00122     if ( size_s <= 0 ) { throw std::runtime_error( "Error during formatting." ); }
00123     auto size = static_cast<size_t>( size_s );
00124     std::unique_ptr<char[]> buf( new char[ size ] );
00125     std::snprintf( buf.get(), size, format.c_str(), args ... );
00126     res.second = std::string( buf.get(), buf.get() + size - 1 ); // We don't want the '\0' inside
00127     return res;
00128 };
00129
00136 Result MultivariateZeroMeanGaussianSampler(Eigen::VectorXd & sample, Eigen::MatrixXd &
covariance_matrix)
00137 {
00138     std::random_device rd;
00139     std::mt19937 gen(rd());
00140     std::normal_distribution<double> dist(0.0, 1.0);
00141
00142     // Verify dimensions
00143     if (covariance_matrix.rows() != covariance_matrix.cols()) return ResultFormat(false,
"covariance_matrix should be square (%li x %li)", covariance_matrix.rows(), covariance_matrix.cols());
00144
00145     // Compute Cholesky Decomposition (LLT)
00146     Eigen::LLT<Eigen::MatrixXd> llt(covariance_matrix);
00147     Eigen::MatrixXd L = llt.matrixL();
00148
00149     // Generate independent standard normal vector Z
00150     Eigen::VectorXd Z = Eigen::VectorXd::NullaryExpr(covariance_matrix.rows(), [&]() { return
dist(gen); });

```

```

00151
00152     // Transform to Multivariate Normal
00153     sample = L * Z;
00154
00155     return ResultFormat(true, "");
00156 }
00157
00165 Result MultivariateGaussianSampler(Eigen::VectorXd & sample, Eigen::VectorXd & mean_vector,
Eigen::MatrixXd & covariance_matrix)
00166 {
00167     // Verify dimensions
00168     if (mean_vector.rows() != covariance_matrix.rows()) return ResultFormat(false, "mean_vector
number of lines (%li) different of covariance_matrix number of lines
(%li)", mean_vector.rows(), covariance_matrix.rows());
00169     // Covariance Matrix will be checked at zero mean Gaussian sampler
00170
00171     // Generate sample from zero mean sample:
00172     Result res = MultivariateZeroMeanGaussianSampler(sample, covariance_matrix);
00173     if (!res.first) return res;
00174
00175     sample += mean_vector;
00176
00177     return res;
00178 }
00179
00186 double UnivariateGaussianSampler(double mean, double variance)
00187 {
00188     std::random_device rd;
00189     std::mt19937 gen(rd());
00190     std::normal_distribution<double> dist(0.0, 1.0);
00191
00192     return mean + sqrt(variance) * dist(gen);
00193 }
00194
00201 double UnivariateUniformSampler(double min_value, double max_value)
00202 {
00203     std::random_device rd;
00204     std::mt19937 gen(rd());
00205     std::uniform_real_distribution<double> dist(0.0, 1.0);
00206
00207     return dist(gen);
00208 }
00209
00210 /*****
00211  * Linear model class:
00212  *****/
00235 class StateSpaceLinearModel
00236 {
00237 public:
00238     Eigen::MatrixXd A;
00239     Eigen::MatrixXd B;
00240     Eigen::MatrixXd C;
00241     Eigen::MatrixXd Q;
00242     Eigen::MatrixXd R;
00243     Eigen::VectorXd X0;
00244     Eigen::MatrixXd P0;
00245     int Nx;
00246     int Nu;
00247     int Ny;
00248     int Nw;
00249     int Nv;
00250     std::vector<bool> X_isangle;
00251     std::vector<bool> Y_isangle;
00252
00261 void SetDimensions(int _Nx, int _Nu, int _Ny, int _Nw, int _Nv)
00262 {
00263     Nx = _Nx; Nu = _Nu; Ny = _Ny; Nw = _Nw; Nv = _Nv;
00264     A.resize(Nx, Nx);
00265     B.resize(Nx, Nu);
00266     C.resize(Ny, Nx);
00267     Q.resize(Nx, Nx);
00268     R.resize(Ny, Ny);
00269     X0.resize(Nx);
00270     P0.resize(Nx, Nx);
00271     X_isangle.resize(Nx);
00272     std::fill(X_isangle.begin(), X_isangle.end(), false);
00273     Y_isangle.resize(Ny);
00274     std::fill(Y_isangle.begin(), Y_isangle.end(), false);
00275 }
00276
00283 Result ProcessModel(Eigen::VectorXd & X, Eigen::VectorXd & U, Eigen::VectorXd & W)
00284 {
00285     Eigen::VectorXd Xpos;
00286     Xpos = A * X + B * U + W; // By using an auxiliary variable, it avoids any risk of copy in
place
00287     X = Xpos;
00288     return BayesianFiltering::ResultFormat(true, "");

```

```

00289     }
00290
00297     Result MeasurementModel(Eigen::VectorXd & Y, Eigen::VectorXd & X, Eigen::VectorXd & V)
00298     {
00299         Y = C * X + V;
00300         return BayesianFiltering::ResultFormat(true, "");
00301     }
00302
00303 };
00304
00305 /*****
00306  * Nonlinear model base class:
00307  *****/
00331 class StateSpaceNonlinearModelBase
00332 {
00333 public:
00334     Eigen::MatrixXd dfdX;
00335     Eigen::MatrixXd dfdW;
00336     Eigen::MatrixXd dhdX;
00337     Eigen::MatrixXd dhdV;
00338     Eigen::MatrixXd Q;
00339     Eigen::MatrixXd R;
00340     Eigen::VectorXd X0;
00341     Eigen::MatrixXd P0;
00342     int Nx;
00343     int Nu;
00344     int Ny;
00345     int Nw;
00346     int Nv;
00347     std::vector<bool> X_isangle;
00348     std::vector<bool> Y_isangle;
00349
00350     // Always provide a virtual destructor if your class has virtual functions!
00351     virtual ~StateSpaceNonlinearModelBase() = default;
00352
00361     void SetDimensions(int _Nx, int _Nu, int _Ny, int _Nw, int _Nv)
00362     {
00363         Nx = _Nx; Nu = _Nu; Ny = _Ny; Nw = _Nw; Nv = _Nv;
00364         dfdX.resize(Nx, Nx);
00365         dfdW.resize(Nx, Nw);
00366         dhdX.resize(Ny, Nx);
00367         dhdV.resize(Ny, Nv);
00368         Q.resize(Nw, Nw);
00369         R.resize(Nv, Nv);
00370         X0.resize(Nx);
00371         P0.resize(Nx, Nx);
00372         X_isangle.resize(Nx);
00373         std::fill(X_isangle.begin(), X_isangle.end(), false);
00374         Y_isangle.resize(Ny);
00375         std::fill(Y_isangle.begin(), Y_isangle.end(), false);
00376     }
00377
00385     virtual Result ProcessModel(Eigen::VectorXd & X, Eigen::VectorXd & U, double t,
Eigen::VectorXd & W)=0; // Deve ser definido pelo usuario
00393     virtual Result MeasurementModel(Eigen::VectorXd & Y, Eigen::VectorXd & X, double t,
Eigen::VectorXd & V)=0; // Deve ser definido pelo usuario
00401     virtual Result UpdatedfdX(Eigen::VectorXd & X, Eigen::VectorXd & U, double t, Eigen::VectorXd
& W)=0; // Deve ser definido pelo usuario
00409     virtual Result UpdatedfdW(Eigen::VectorXd & X, Eigen::VectorXd & U, double t, Eigen::VectorXd
& W)=0; // Deve ser definido pelo usuario
00416     virtual Result UpdateddhdX(Eigen::VectorXd & X, double t, Eigen::VectorXd & V)=0; // Deve ser
definido pelo usuario
00423     virtual Result UpdateddhdV(Eigen::VectorXd & X, double t, Eigen::VectorXd & V)=0; // Deve ser
definido pelo usuario
00424 };
00425
00426 /*****
00427  * Bayesian filter base class:
00428  *****/
00445 class BayesianFilter
00446 {
00447 public:
00448     Eigen::VectorXd X;
00449     Eigen::MatrixXd P;
00450
00451     Eigen::VectorXd Y_predicted;
00452     Eigen::VectorXd V_predicted;
00453     Eigen::MatrixXd S_predicted;
00454     double d2_predicted;
00455
00456     Eigen::VectorXd Y_corrected;
00457     Eigen::VectorXd V_corrected;
00458     Eigen::MatrixXd S_corrected;
00459     double d2_corrected;
00460
00461     BayesianFilter()
00462     {

```

```

00463         d2_predicted = 0;
00464         d2_corrected = 0;
00465     }
00466     virtual ~BayesianFilter() {}
00467
00468     protected: // Only derived classes have access
00469
00470     void Reset(Eigen::VectorXd &X0, Eigen::MatrixXd &P0, int Nx, int Ny)
00471     {
00472
00473         X.resize(Nx, 1);
00474         P.resize(Nx, Nx);
00475         Y_predicted.resize(Ny, 1);
00476         V_predicted.resize(Ny, 1);
00477         S_predicted.resize(Ny, Ny);
00478         Y_corrected.resize(Ny, 1);
00479         V_corrected.resize(Ny, 1);
00480         S_corrected.resize(Ny, Ny);
00481
00482         d2_predicted = 0;
00483         d2_corrected = 0;
00484         X = X0;
00485         P = P0;
00486     }
00487 };
00488
00489 class LinearBayesianFilter : public BayesianFilter
00490 {
00491 public:
00492     virtual ~LinearBayesianFilter() {std::cout << "\nLinearBayesianFilter Base destroyed\n";}
00493
00494     virtual Result Prediction(Eigen::VectorXd & U, StateSpaceLinearModel& linear_model)=0;
00495     virtual Result Correction(Eigen::VectorXd & Y, StateSpaceLinearModel& linear_model)=0;
00496     virtual void Reset(StateSpaceLinearModel& linear_model)=0;
00497
00498     protected:
00499     Result CheckModel(StateSpaceLinearModel& linear_model)
00500     {
00501         if (linear_model.A.rows() != X.rows()) return ResultFormat(false, "Number of rows of A should
be the same number of rows of X");
00502         if (linear_model.A.cols() != X.rows()) return ResultFormat(false, "Number of cols of A should
be the same number of rows of X");
00503         if (linear_model.B.rows() != X.rows()) return ResultFormat(false, "Number of rows of B should
be the same number of rows of X");
00504         return ResultFormat(true, "");
00505     }
00506 };
00507
00508 class KalmanFilter : public LinearBayesianFilter
00509 {
00510 public:
00511     virtual ~KalmanFilter() {std::cout << "\nKalmanFilter derived class destroyed\n";}
00512
00513     void Reset(StateSpaceLinearModel& linear_model) override
00514     {
00515         BayesianFilter::Reset(linear_model.X0, linear_model.P0, linear_model.Nx, linear_model.Ny);
00516     }
00517
00518     Result Prediction(Eigen::VectorXd & U, StateSpaceLinearModel& linear_model) override
00519     {
00520         Result res = CheckModel(linear_model); if(!res.first) return res;
00521
00522         // Predicao
00523         X = linear_model.A * X + linear_model.B * U;
00524         P = linear_model.A * P * linear_model.A.transpose() + linear_model.Q;
00525         return ResultFormat(true, "");
00526     }
00527
00528     Result Correction(Eigen::VectorXd & Y, StateSpaceLinearModel& linear_model) override
00529     {
00530         Result res = CheckModel(linear_model); if(!res.first) return res;
00531
00532         // Correcao
00533         Y_predicted = linear_model.C * X;
00534         V_predicted = Y - Y_predicted;
00535         for(int i=0; i < V_predicted.size(); ++i){
00536             if(linear_model.Y_isangle[i]) V_predicted(i) =
atan2(sin(V_predicted(i)),cos(V_predicted(i)));
00537         }
00538         S_predicted = linear_model.C * P * linear_model.C.transpose() + linear_model.R;
00539         d2_predicted = (V_predicted.transpose() * S_predicted.inverse() * V_predicted)(0, 0);
00540
00541         Eigen::MatrixXd G = P * linear_model.C.transpose() * S_predicted.inverse();
00542         Eigen::MatrixXd Gi = G * V_predicted;
00543         for(int i=0; i < Gi.rows(); ++i){
00544             if(linear_model.X_isangle[i]) Gi(i) = atan2(sin(Gi(i)),cos(Gi(i)));
00545         }
00546     }

```

```

00575         X = X + Gi;
00576         //P = (Eigen::MatrixXd::Identity(X.rows(), X.rows()) - G * linear_model.C) * P;
00577         // More numerically stable P update:
00578         P = (Eigen::MatrixXd::Identity(X.rows(), X.rows()) - G * linear_model.C) * P
            * ((Eigen::MatrixXd::Identity(X.rows(), X.rows()) - G * linear_model.C).transpose() +
              G*linear_model.R*G.transpose());
00579
00580         Y_corrected = linear_model.C * X;
00581         V_corrected = Y - Y_corrected;
00582         for(int i=0; i < V_corrected.size(); ++i){
00583             if(linear_model.Y_isangle[i]) V_corrected(i) =
00584                 atan2(sin(V_corrected(i)), cos(V_corrected(i)));
00585         }
00586         S_corrected = linear_model.C * P * linear_model.C.transpose() + linear_model.R;
00587         d2_corrected = (V_corrected.transpose() * S_corrected.inverse() * V_corrected)(0, 0);
00588         return ResultFormat(true, "");
00589     }
00590 };
00591
00592 class AdaptiveKalmanFilter : public LinearBayesianFilter
00593 {
00594 public:
00595     double alpha;
00596     double beta;
00597
00598     virtual ~AdaptiveKalmanFilter() {std::cout << "\nAdaptiveKalmanFilter derived class
00599     destroyed\n";}
00600
00601     void Reset(StateSpaceLinearModel& linear_model, double _alpha, double _beta)
00602     {
00603         alpha = _alpha;
00604         beta = _beta;
00605         BayesianFilter::Reset(linear_model.X0, linear_model.P0, linear_model.Nx, linear_model.Ny);
00606     }
00607
00608     void Reset(StateSpaceLinearModel& linear_model) override
00609     {
00610         Reset(linear_model, 0.95, 0.99);
00611     }
00612
00613     Result Prediction(Eigen::VectorXd & U, StateSpaceLinearModel& linear_model) override
00614     {
00615         Result res = CheckModel(linear_model); if(!res.first) return res;
00616         // Predicao
00617         X = linear_model.A * X + linear_model.B * U;
00618         P = linear_model.A * P * linear_model.A.transpose() + linear_model.Q;
00619         return ResultFormat(true, "");
00620     }
00621
00622     Result Correction(Eigen::VectorXd & Y, StateSpaceLinearModel& linear_model) override
00623     {
00624         Result res = CheckModel(linear_model); if(!res.first) return res;
00625
00626         // Innovation
00627         Y_predicted = linear_model.C * X;
00628         V_predicted = Y - Y_predicted;
00629         for(int i=0; i < V_predicted.size(); ++i){
00630             if(linear_model.Y_isangle[i]) V_predicted(i) =
00631                 atan2(sin(V_predicted(i)), cos(V_predicted(i)));
00632         }
00633         S_predicted = linear_model.C * P * linear_model.C.transpose() + linear_model.R;
00634         d2_predicted = (V_predicted.transpose() * S_predicted.inverse() * V_predicted)(0, 0);
00635         Eigen::MatrixXd P_predicted = P;
00636
00637         // State update
00638         Eigen::MatrixXd G = P * linear_model.C.transpose() * S_predicted.inverse();
00639         Eigen::MatrixXd Gi = G * V_predicted;
00640         for(int i=0; i < Gi.rows(); ++i){
00641             if(linear_model.X_isangle[i]) Gi(i) = atan2(sin(Gi(i)), cos(Gi(i)));
00642         }
00643         X = X + Gi;
00644         //P = (Eigen::MatrixXd::Identity(X.rows(), X.rows()) - G * linear_model.C) * P;
00645         // More numerically stable P update:
00646         P = (Eigen::MatrixXd::Identity(X.rows(), X.rows()) - G * linear_model.C) * P
            * ((Eigen::MatrixXd::Identity(X.rows(), X.rows()) - G * linear_model.C).transpose() +
              G*linear_model.R*G.transpose());
00647
00648         // Innovation corrected
00649         Y_corrected = linear_model.C * X;
00650         V_corrected = Y - Y_corrected;
00651         for(int i=0; i < V_corrected.size(); ++i){
00652             if(linear_model.Y_isangle[i]) V_corrected(i) =
00653                 atan2(sin(V_corrected(i)), cos(V_corrected(i)));
00654         }
00655         S_corrected = linear_model.C * P * linear_model.C.transpose() + linear_model.R;

```

```

00679         d2_corrected = (V_corrected.transpose() * S_corrected.inverse() * V_corrected)(0, 0);
00680
00681         // R and Q updates
00682         linear_model.R = alpha * linear_model.R + (1.0-alpha)*(V_predicted *
V_predicted.transpose() + linear_model.C * P_predicted * linear_model.C.transpose());
00683         linear_model.Q = beta * linear_model.Q + (1.0-beta)*(G * V_predicted *
V_predicted.transpose() * G.transpose());
00684
00685         return ResultFormat(true, "");
00686     }
00687 };
00688
00689 class RobustSelfAdaptiveKalmanFilter : public LinearBayesianFilter
00690 {
00691     Eigen::MatrixXd P_corrected;
00692
00693 public:
00694     virtual ~RobustSelfAdaptiveKalmanFilter() {std::cout << "\nRobustSelfAdaptiveKalmanFilter
derived class destroyed\n";}
00695
00696     void Reset(StateSpaceLinearModel& linear_model) override
00697     {
00698         BayesianFilter::Reset(linear_model.X0, linear_model.P0, linear_model.Nx, linear_model.Ny);
00699         P_corrected = linear_model.P0;
00700     }
00701
00702     Result Prediction(Eigen::VectorXd & U, StateSpaceLinearModel& linear_model) override
00703     {
00704         Result res = CheckModel(linear_model); if(!res.first) return res;
00705         // Predicao
00706         X = linear_model.A * X + linear_model.B * U;
00707         P = linear_model.A * P * linear_model.A.transpose() + linear_model.Q;
00708         return ResultFormat(true, "");
00709     }
00710
00711     Result Correction(Eigen::VectorXd & Y, StateSpaceLinearModel& linear_model) override
00712     {
00713         Result res = CheckModel(linear_model); if(!res.first) return res;
00714
00715         // Ohmega calculation
00716         double sigma = linear_model.R(0,0);
00717         double epsilon = abs(V_corrected(0,0));
00718         double ohmega = 0.0;
00719         if (sigma > 0.0){
00720             ohmega = (epsilon <= sigma) ? 1.0 : sigma/epsilon;
00721         }
00722
00723         // R update
00724         linear_model.R = ohmega * linear_model.R + (1-ohmega)*(V_corrected *
V_corrected.transpose() + linear_model.C * P_corrected * linear_model.C.transpose());
00725
00726         // Innovation
00727         Y_predicted = linear_model.C * X;
00728         V_predicted = Y - Y_predicted;
00729         for(int i=0; i < V_predicted.size(); ++i){
00730             if(linear_model.Y_isangle[i]) V_predicted(i) =
atan2(sin(V_predicted(i)),cos(V_predicted(i)));
00731         }
00732         S_predicted = linear_model.C * P * linear_model.C.transpose() + linear_model.R;
00733         d2_predicted = (V_predicted.transpose() * S_predicted.inverse() * V_predicted)(0, 0);
00734
00735         // State update
00736         Eigen::MatrixXd G = P * linear_model.C.transpose() * S_predicted.inverse();
00737         Eigen::MatrixXd Gi = G * V_predicted;
00738         for(int i=0; i < Gi.rows(); ++i){
00739             if(linear_model.X_isangle[i]) Gi(i) = atan2(sin(Gi(i)),cos(Gi(i)));
00740         }
00741         X = X + Gi;
00742         //P = (Eigen::MatrixXd::Identity(X.rows(), X.rows()) - G * linear_model.C) * P;
00743         // More numerically stable P update:
00744         P = (Eigen::MatrixXd::Identity(X.rows(), X.rows()) - G * linear_model.C) * P
*((Eigen::MatrixXd::Identity(X.rows(), X.rows()) - G * linear_model.C).transpose() +
G*linear_model.R*G.transpose());
00745
00746         // Innovation corrected
00747         Y_corrected = linear_model.C * X;
00748         V_corrected = Y - Y_corrected;
00749         for(int i=0; i < V_corrected.size(); ++i){
00750             if(linear_model.Y_isangle[i]) V_corrected(i) =
atan2(sin(V_corrected(i)),cos(V_corrected(i)));
00751         }
00752         S_corrected = linear_model.C * P * linear_model.C.transpose() + linear_model.R;
00753         d2_corrected = (V_corrected.transpose() * S_corrected.inverse() * V_corrected)(0, 0);
00754
00755         // Q update

```

```

00779         linear_model.Q = ohmega * linear_model.Q + (1-ohmega)*(G * V_corrected *
V_corrected.transpose() * G.transpose());
00780
00781         // P corrected, to be used in next interaction
00782         P_corrected = P;
00783
00784         return ResultFormat(true, "");
00785     }
00786 };
00787
00788 class NonlinearBayesianFilter : public BayesianFilter
00789 {
00790 public:
00791     virtual ~NonlinearBayesianFilter() {std::cout << "\nNonlinearBayesianFilter Base destroyed\n";}
00792     virtual Result Prediction(Eigen::VectorXd & U, double t, StateSpaceNonlinearModelBase&
nonlinear_model)=0;
00793     virtual Result Correction(Eigen::VectorXd & Y, double t, StateSpaceNonlinearModelBase&
nonlinear_model)=0;
00794     virtual void Reset(StateSpaceNonlinearModelBase& nonlinear_model)=0;
00795 };
00796
00797 class ExtendedKalmanFilter : public NonlinearBayesianFilter
00798 {
00799 public:
00800     virtual ~ExtendedKalmanFilter() {std::cout << "\nExtendedKalmanFilter derived class
destroyed\n";}
00801     void Reset(StateSpaceNonlinearModelBase& nonlinear_model) override
00802     {
00803         BayesianFilter::Reset(nonlinear_model.X0, nonlinear_model.P0, nonlinear_model.Nx,
nonlinear_model.Ny);
00804     }
00805     Result Prediction(Eigen::VectorXd & U, double t, StateSpaceNonlinearModelBase&
nonlinear_model) override
00806     {
00807         // Update model (linearized on previous correction state)
00808         Eigen::VectorXd W = Eigen::MatrixXd::Zero(nonlinear_model.dfdW.rows(), 1);
00809         nonlinear_model.UpdatedfdX(X,U,t,W);
00810         nonlinear_model.UpdatedfdW(X,U,t,W);
00811
00812         // Predicao
00813         Result res = nonlinear_model.ProcessModel(X,U,t,W);
00814         if(!res.first) return res;
00815         P = nonlinear_model.dfdX * P * nonlinear_model.dfdX.transpose() +
nonlinear_model.dfdW*nonlinear_model.Q*nonlinear_model.dfdW.transpose();
00816
00817         return res;
00818     }
00819     Result Correction(Eigen::VectorXd & Y, double t, StateSpaceNonlinearModelBase&
nonlinear_model) override
00820     {
00821         Result res;
00822
00823         // Update model (linearized on predicted state)
00824
00825         Eigen::VectorXd V = Eigen::VectorXd::Zero(nonlinear_model.dhdV.rows(), 1);
00826         nonlinear_model.UpdatedhdX(X,t,V);
00827         nonlinear_model.UpdatedhdV(X,t,V);
00828
00829         // State correction:
00830         res = nonlinear_model.MeasurementModel(Y_predicted,X,t,V);
00831         if(!res.first) return res;
00832
00833         V_predicted = Y - Y_predicted;
00834         for(int i=0; i < V_predicted.size(); ++i){
00835             if(nonlinear_model.Y_isangle[i]) V_predicted(i) =
atan2(sin(V_predicted(i)),cos(V_predicted(i)));
00836         }
00837         S_predicted = nonlinear_model.dhdX * P * nonlinear_model.dhdX.transpose() +
nonlinear_model.dhdV * nonlinear_model.R * nonlinear_model.dhdV.transpose();
00838         d2_predicted = (V_predicted.transpose() * S_predicted.inverse() * V_predicted)(0, 0);
00839
00840         Eigen::MatrixXd G = P * nonlinear_model.dhdX.transpose() * S_predicted.inverse();
00841         Eigen::MatrixXd Gi = G * V_predicted;
00842         for(int i=0; i < Gi.rows(); ++i){
00843             if(nonlinear_model.X_isangle[i]) Gi(i) = atan2(sin(Gi(i)),cos(Gi(i)));
00844         }
00845         X = X + Gi;
00846         // P = (Eigen::MatrixXd::Identity(X.rows(), X.rows()) - G * nonlinear_model.dhdX) * P;
00847         // More numerically stable P update:
00848         P = (Eigen::MatrixXd::Identity(X.rows(), X.rows()) - G * nonlinear_model.dhdX) * P
+ ((Eigen::MatrixXd::Identity(X.rows(), X.rows()) - G * nonlinear_model.dhdX).transpose() +
G*nonlinear_model.R*G.transpose());

```

```

00878
00879 // Update model (linearized on corrected state)
00880 V = Eigen::VectorXd::Zero(nonlinear_model.dhdV.rows(), 1);
00881 nonlinear_model.UpdatedhdX(X,t,V);
00882 nonlinear_model.UpdatedhdV(X,t,V);
00883
00884 // Generate corrected output, innovation and d^2:
00885 res = nonlinear_model.MeasurementModel(Y_corrected,X,t,V);
00886 if(!res.first) return res;
00887 V_corrected = Y - Y_corrected;
00888 for(int i=0; i < V_corrected.size(); ++i){
00889     if(nonlinear_model.Y_isangle[i]) V_corrected(i) =
atan2(sin(V_corrected(i)),cos(V_corrected(i)));
00890 }
00891 S_corrected = nonlinear_model.dhdX * P * nonlinear_model.dhdX.transpose() +
nonlinear_model.dhdV * nonlinear_model.R * nonlinear_model.dhdV.transpose();
00892 d2_corrected = (V_corrected.transpose() * S_corrected.inverse() * V_corrected)(0, 0);
00893
00894 return res;
00895 }
00896 };
00897 };
00898
00904 class UnscentedKalmanFilter : public NonlinearBayesianFilter
00905 {
00906 private:
00913 void InitializeWeights(Eigen::VectorXd & Wsigma, double & kappa, int Nx)
00914 {
00915     int Nsigma = 2*Nx + 1;
00916
00917     Wsigma.resize(Nsigma,1);
00918
00919     kappa = (3 > Nx-3) ? 3 : Nx-3;
00920     Wsigma(0) = kappa/(Nx+kappa);
00921     for (int i = 1; i < Wsigma.rows(); ++i) {
00922         Wsigma(i) = 1/(2.0*(Nx+kappa));
00923     }
00924     Wsigma = Wsigma / Wsigma.sum(); // Garante soma 1.
00925 }
00926
00927 public:
00928 // Override virtual members:
00929 virtual ~UnscentedKalmanFilter() {std::cout << "\nUnscentedKalmanFilter derived class
destroyed\n";}
00930
00935 void Reset(StateSpaceNonlinearModelBase& nonlinear_model) override
00936 {
00937     BayesianFilter::Reset(nonlinear_model.X0, nonlinear_model.P0, nonlinear_model.Nx,
nonlinear_model.Ny);
00938 }
00939
00946 Result Prediction(Eigen::VectorXd & U, double t, StateSpaceNonlinearModelBase&
nonlinear_model) override
00947 {
00948     Result res = BayesianFiltering::ResultFormat(true,"");
00949
00950 // Prediction
00951 // The function x(k) = f(x(k-1),u(k),w(k),t(k)) is converted to x(k) = f(xa(k),u(k),t(k))
with xa(k) = [x(k-1); w(k)]
00952 int Nx = nonlinear_model.Nx;
00953 int Nw = nonlinear_model.Nw;
00954 int DimX = Nx + Nw;
00955 int DimY = Nx;
00956
00957 Eigen::VectorXd Wsigma;
00958 double kappa;
00959 InitializeWeights(Wsigma, kappa, DimX);
00960 int Nsigma = Wsigma.rows();
00961
00962 // Generate augmented Xa and Pa
00963 Eigen::VectorXd Xa(DimX);
00964 Xa << X, Eigen::VectorXd::Zero(Nw);
00965
00966 Eigen::MatrixXd Pa(DimX,DimX);
00967 Pa << P, Eigen::MatrixXd::Zero(Nx,Nw),
Eigen::MatrixXd::Zero(Nw, Nx), nonlinear_model.Q;
00968
00969 // Generate sigma points:
00970 Eigen::MatrixXd Xsigma(DimX,Nsigma);
00971 Eigen::MatrixXd Ysigma(DimY,Nsigma);
00972
00973 Eigen::LLT<Eigen::MatrixXd> llt((DimX+kappa)*Pa);
00974 Eigen::MatrixXd L = llt.matrixL();
00975
00976 Eigen::VectorXd Xi(Nx);
00977 Eigen::VectorXd Wi(Nw);
00978
00979

```

```

00980         for (int i = 0; i < Nsigma; ++i) {
00981             if (i==0){
00982                 Xsigma.col(i) = Xa;
00983             }
00984             else if (i <= DimX){
00985                 Xsigma.col(i) = Xa + L.col(i-1);
00986             }
00987             else{
00988                 Xsigma.col(i) = Xa - L.col(i-1-DimX);
00989             }
00990             Xi = Xsigma.block(0 , i, Nx, 1);
00991             Wi = Xsigma.block(Nx, i, Nw, 1);
00992             Result res = nonlinear_model.ProcessModel(Xi,U,t,Wi);
00993             if(!res.first) return res;
00994             Ysigma.col(i) = Xi;
00995         }
00996         // Compute prediction
00997         X = Eigen::VectorXd::Zero(Nx);
00998         for (int i = 0; i < Nsigma; ++i) {
00999             X += Wsigma(i)*Ysigma.col(i);
01000         }
01001         P = Eigen::MatrixXd::Zero(Nx,Nx);
01002         for (int i = 0; i < Nsigma; ++i) {
01003             P += Wsigma(i)*(Ysigma.col(i)-X)*(Ysigma.col(i)-X).transpose();
01004         }
01005
01006         return res;
01007     }
01008
01015     Result Correction(Eigen::VectorXd & Y, double t, StateSpaceNonlinearModelBase&
nonlinear_model) override
01016     {
01017         Result res = BayesianFiltering::ResultFormat(true,"");
01018
01019         // Correction
01020         // The function y(k) = h(x(k),v(k)) is converted to x(k) = h(xa(k)) with xa(k) = [x(k);
v(k)]
01021         int Nx = nonlinear_model.Nx;
01022         int Ny = nonlinear_model.Ny;
01023         int Nv = nonlinear_model.Nv;
01024         int DimX = Nx + Nv;
01025         int DimY = Ny;
01026
01027         Eigen::VectorXd Wsigma;
01028         double kappa;
01029         InitializeWeights(Wsigma, kappa, DimX);
01030         int Nsigma = Wsigma.rows();
01031
01032         // Generate augmented Xa and Pa
01033         Eigen::VectorXd Xa(DimX);
01034         Xa << X, Eigen::VectorXd::Zero(Nv);
01035
01036         Eigen::MatrixXd Pa(DimX,DimX);
01037         Pa << P, Eigen::MatrixXd::Zero(Nx,Nv),
Eigen::MatrixXd::Zero(Nv,Nx), nonlinear_model.R;
01038
01039         // Generate sigma points:
01040         Eigen::MatrixXd Xsigma(DimX,Nsigma); Xsigma = Eigen::MatrixXd::Zero(DimX,Nsigma);
01041         Eigen::MatrixXd Ysigma(DimY,Nsigma); Ysigma = Eigen::MatrixXd::Zero(DimY,Nsigma);
01042
01043         Eigen::MatrixXd L;
01044         {
01045             Eigen::LLT<Eigen::MatrixXd> llt((DimX+kappa)*Pa);
01046             L = llt.matrixL();
01047         }
01048
01049         Eigen::VectorXd Xi(Nx);
01050         Eigen::VectorXd Vi(Nv);
01051         Eigen::VectorXd Yi(Ny);
01052         for (int i = 0; i < Nsigma; ++i) {
01053             if (i==0){
01054                 Xsigma.col(i) = Xa;
01055             }
01056             else if (i <= DimX){
01057                 Xsigma.col(i) = Xa + L.col(i-1);
01058             }
01059             else{
01060                 Xsigma.col(i) = Xa - L.col(i-1-DimX);
01061             }
01062             Xi = Xsigma.block(0 , i, Nx, 1);
01063             Vi = Xsigma.block(Nx, i, Nv, 1);
01064             res = nonlinear_model.MeasurementModel(Yi,Xi,t,Vi); if(!res.first) return res;
01065             Ysigma.col(i) = Yi;
01066         }
01067         // Compute correction
01068         Y_predicted = Eigen::VectorXd::Zero(Ny);
01069         for (int i = 0; i < Nsigma; ++i) {

```

```

01071         Y_predicted += Wsigma(i)*Ysigma.col(i);
01072     }
01073     Eigen::MatrixXd Pyy = Eigen::MatrixXd::Zero(Nx,Nx);
01074     Eigen::MatrixXd Pxy = Eigen::MatrixXd::Zero(Nx,Ny);
01075     for (int i = 0; i < Nsigma; ++i) {
01076         Eigen::VectorXd dX = Xsigma.block(0 , i, Nx, 1)-X;
01077         for(int j=0; j < dX.rows(); ++j){
01078             if(nonlinear_model.X_isangle[j]) dX(j) = atan2(sin(dX(j)),cos(dX(j)));
01079         }
01080         Eigen::VectorXd dY = Ysigma.col(i)-Y_predicted;
01081         for(int j=0; j < dY.rows(); ++j){
01082             if(nonlinear_model.Y_isangle[j]) dY(j) = atan2(sin(dY(j)),cos(dY(j)));
01083         }
01084         Pyy += Wsigma(i)*(dY)*(dY.transpose());
01085         Pxy += Wsigma(i)*(dX)*(dY.transpose());
01086     }
01087     V_predicted = Y - Y_predicted;
01088     for(int i=0; i < V_predicted.size(); ++i){
01089         if(nonlinear_model.Y_isangle[i]) V_predicted(i) =
01090         atan2(sin(V_predicted(i)),cos(V_predicted(i)));
01091     }
01092     S_predicted = Pyy;
01093     d2_predicted = (V_predicted.transpose() * S_predicted.inverse() * V_predicted)(0, 0);
01094
01095     Eigen::MatrixXd G = Pxy * Pyy.inverse();
01096     Eigen::MatrixXd Gi = G * V_predicted;
01097     for(int i=0; i < Gi.rows(); ++i){
01098         if(nonlinear_model.X_isangle[i]) Gi(i) = atan2(sin(Gi(i)),cos(Gi(i)));
01099     }
01100     X = X + Gi;
01101     P = P - G*Pyy*G.transpose();
01102
01103     // Compute corrected output, innovation and d^2:
01104     Xa << X, Eigen::VectorXd::Zero(Nv);
01105     Pa << P, Eigen::MatrixXd::Zero(Nx,Nv),
01106         Eigen::MatrixXd::Zero(Nv,Nx), nonlinear_model.R;
01107
01108     {
01109         Eigen::LLT<Eigen::MatrixXd> llt((DimX+kappa)*Pa);
01110         L = llt.matrixL();
01111     }
01112     for (int i = 0; i < Nsigma; ++i) {
01113         if (i==0){
01114             Xsigma.col(i) = Xa;
01115         }
01116         else if (i <= DimX){
01117             Xsigma.col(i) = Xa + L.col(i-1);
01118         }
01119         else{
01120             Xsigma.col(i) = Xa - L.col(i-1-DimX);
01121         }
01122         Xi = Xsigma.block(0 , i, Nx, 1);
01123         Vi = Xsigma.block(Nx, i, Ny, 1);
01124         res = nonlinear_model.MeasurementModel(Yi,Xi,t,Vi); if(!res.first) return res;
01125         Ysigma.col(i) = Yi;
01126     }
01127     Y_corrected = Eigen::VectorXd::Zero(Ny);
01128     for (int i = 0; i < Nsigma; ++i) {
01129         Y_corrected += Wsigma(i)*Ysigma.col(i);
01130     }
01131     Pyy = Eigen::MatrixXd::Zero(Nx,Nx);
01132     for (int i = 0; i < Nsigma; ++i) {
01133         Pyy +=
01134         Wsigma(i)*(Ysigma.col(i)-Y_corrected)*((Ysigma.col(i)-Y_corrected).transpose());
01135     }
01136     V_corrected = Y - Y_corrected;
01137     for(int i=0; i < V_corrected.size(); ++i){
01138         if(nonlinear_model.Y_isangle[i]) V_corrected(i) =
01139         atan2(sin(V_corrected(i)),cos(V_corrected(i)));
01140     }
01141     S_corrected = Pyy;
01142     d2_corrected = (V_corrected.transpose() * S_corrected.inverse() * V_corrected)(0, 0);
01143
01144     return res;
01145 }
01146 };
01147
01148 class SamplingImportanceResamplingFilter : public NonlinearBayesianFilter
01149 {
01150 private:
01151     void ResetWeights()
01152     {
01153         long int Nparticles = Wparticles.rows();
01154         Wparticles.setConstant(1.0/((double) (Nparticles)));
01155     }
01156 };
01157

```

```

01164     public:
01165         enum class EstimateType
01166         {
01167             MaximumAPosteriori,
01168             ConditionalMean,
01169         };
01170
01171         static inline const char* EnumToString(EstimateType x)
01172         {
01173             const std::map<EstimateType, const char*> Strings {
01174                 { EstimateType::MaximumAPosteriori, "Maximum a Posteriori estimate" },
01175                 { EstimateType::ConditionalMean, "Conditional Mean estimate" },
01176             };
01177             auto it = Strings.find(x);
01178             return it == Strings.end() ? "Out of range" : it->second;
01179         }
01180
01181         // Specific data
01182         EstimateType estimate_type;
01183         long int Nparticles;
01184         Eigen::MatrixXd Xparticles;
01185         Eigen::MatrixXd Yparticles;
01186         Eigen::VectorXd Wparticles;
01187         double ESS;
01188
01189         // Override virtual members:
01190         virtual ~SamplingImportanceResamplingFilter() {std::cout <<
01191             "\nSamplingImportanceResamplingFilter derived class destroyed\n";}
01192
01193         void Reset(StateSpaceNonlinearModelBase& nonlinear_model, long int _Nparticles, EstimateType
01194             _estimate_type)
01195         {
01196             BayesianFilter::Reset(nonlinear_model.X0, nonlinear_model.P0, nonlinear_model.Nx,
01197                 nonlinear_model.Ny);
01198             Nparticles = _Nparticles;
01199             estimate_type = _estimate_type;
01200             Xparticles.resize(nonlinear_model.Nx, Nparticles);
01201             Yparticles.resize(nonlinear_model.Ny, Nparticles);
01202             Wparticles.resize(Nparticles);
01203             ResetWeights();
01204             ESS = Nparticles;
01205
01206             Result res = BayesianFiltering::ResultFormat(true, "");
01207             Eigen::VectorXd Xi(nonlinear_model.Nx);
01208             for (long int i = 0; i < Nparticles; ++i) {
01209                 Xparticles.col(i) = X;
01210                 res = MultivariateZeroMeanGaussianSampler(Xi, P);
01211                 if(res.first){
01212                     Xparticles.col(i) = Xi + X;
01213                 }
01214             }
01215
01216             void Reset(StateSpaceNonlinearModelBase& nonlinear_model) override
01217             {
01218                 Reset(nonlinear_model, 1000, EstimateType::ConditionalMean); // default number of
01219                 particles
01220             }
01221
01222             Result NormalizeWeights()
01223             {
01224                 if (Wparticles.sum() > 0){
01225                     double c = 1.0/Wparticles.sum();
01226                     Wparticles *= c;
01227                 }
01228                 else{
01229                     return ResultFormat(false, "SIR filter diverged: sum of weights = %f",
01230                         Wparticles.sum());
01231                 }
01232                 return BayesianFiltering::ResultFormat(true, "");
01233             }
01234
01235             Result ComputeESS()
01236             {
01237                 double sum = 0;
01238
01239                 for (long int i = 0; i < Wparticles.rows(); ++i) {
01240                     sum += Wparticles(i)*Wparticles(i);
01241                 }
01242                 if (sum <= 0.0){
01243                     return ResultFormat(false, "SIR filter diverged: sum of square weights = %f", sum);
01244                 }
01245                 ESS = 1/sum;
01246                 return BayesianFiltering::ResultFormat(true, "");
01247             }
01248         }
01249     }
01250
01251     Result ComputeESS()
01252     {
01253         double sum = 0;
01254
01255         for (long int i = 0; i < Wparticles.rows(); ++i) {
01256             sum += Wparticles(i)*Wparticles(i);
01257         }
01258         if (sum <= 0.0){
01259             return ResultFormat(false, "SIR filter diverged: sum of square weights = %f", sum);
01260         }
01261         ESS = 1/sum;
01262         return BayesianFiltering::ResultFormat(true, "");
01263     }
01264 }
01265
01266
01267

```

```

01274     Result Prediction(Eigen::VectorXd & U, double t, StateSpaceNonlinearModelBase&
nonlinear_model) override
01275     {
01276         Result res = BayesianFiltering::ResultFormat(true, "");
01277
01278         // Prediction
01279         // Generic nonlinear model  $x(k) = f(x(k-1), u(k), w(k), t(k))$ 
01280         int Nx = nonlinear_model.Nx;
01281         int Nw = nonlinear_model.Nw;
01282         long int Nparticles = Xparticles.cols();
01283
01284         ResetWeights();
01285
01286         Eigen::VectorXd Xi(Nx);
01287         Eigen::VectorXd Wi(Nw);
01288         for (long int i = 0; i < Nparticles; ++i) {
01289             Xi = Xparticles.col(i);
01290             res = MultivariateZeroMeanGaussianSampler(Wi, nonlinear_model.Q);
01291             if(!res.first) return res;
01292             res = nonlinear_model.ProcessModel(Xi, U, t, Wi);
01293             if(!res.first) return res;
01294             Xparticles.col(i) = Xi;
01295         }
01296
01297         // Compute prediction: assuming equal weights, Conditional Mean and Maximum a Posteriori
are the same.
01298         X = Eigen::VectorXd::Zero(Nx);
01299         for (long int i = 0; i < Nparticles; ++i) {
01300             X += Wparticles(i)*Xparticles.col(i);
01301         }
01302         P = Eigen::MatrixXd::Zero(Nx, Nx);
01303         for (long int i = 0; i < Nparticles; ++i) {
01304             P += Wparticles(i)*(Xparticles.col(i)-X)*(Xparticles.col(i)-X).transpose();
01305         }
01306
01307         // Particle Weights
01308         // No need to update;
01309
01310         res = ComputeESS(); if(!res.first) return res;
01311
01312         return res;
01313     }
01314
01321     Result Correction(Eigen::VectorXd & Y, double t, StateSpaceNonlinearModelBase&
nonlinear_model) override
01322     {
01323         Result res = BayesianFiltering::ResultFormat(true, "");
01324
01325         // Correction
01326         // Generic nonlinear model  $y(k) = h(x(k), v(k))$ 
01327         int Nx = nonlinear_model.Nx;
01328         int Ny = nonlinear_model.Ny;
01329         int Nv = nonlinear_model.Nv;
01330         long int Nparticles = Wparticles.rows();
01331
01332         // Update weights
01333         // %% Observar que os pesos q das particulas não são passadas como argumento. De fato,
assume-se que estes pesos
01334         // %% são todos iguais a 1/Ns, decorrente da ultima execução do estimador. Assim, o
calculo da ponderação
01335         // %%  $q(k) = q(k-1)*p(z(k)|x(k))$  seguido da normalização  $q(k) = q(k)/\text{sum}(q)$  dá na mesma
coisa do que fazer
01336         // %%  $q(k) = p(z(k)|x(k))$  seguido de  $q(k) = q(k)/\text{sum}(q)$ , como está implementado.
01337         Eigen::VectorXd Xi(Nx);
01338         Eigen::VectorXd Vi(Nv);
01339         Eigen::VectorXd Yi(Ny);
01340         Y_predicted = Eigen::VectorXd::Zero(Ny); // Computer with weights equal, leading to the
mean
01341         for (long int i = 0; i < Nparticles; ++i) {
01342             Xi = Xparticles.col(i);
01343             res = MultivariateZeroMeanGaussianSampler(Vi, nonlinear_model.R); if(!res.first)
return res;
01344             res = nonlinear_model.MeasurementModel(Yi, Xi, t, Vi); if(!res.first) return res;
01345             Y_predicted += Yi; // Simple mean
01346             Wparticles(i) = exp((-0.5)*(Y-Yi).transpose()*(nonlinear_model.R.inverse())*(Y-Yi));
01347
01348             Yparticles.col(i) = Yi;
01349         }
01350         Y_predicted /= ((double)(Nparticles));
01351         res = NormalizeWeights(); if(!res.first) return res;
01352
01353         res = ComputeESS(); if(!res.first) return res;
01354
01355         S_predicted = Eigen::MatrixXd::Zero(Ny, Ny);
01356         for (int i = 0; i < Nparticles; ++i) {
01357             V_predicted = Yparticles.col(i)-Y_predicted;
01358             for(int j=0; j < V_predicted.size(); ++j){

```

```

01358         if(nonlinear_model.Y_isangle[j]) V_predicted(j) =
atan2(sin(V_predicted(j)),cos(V_predicted(j)));
01359     }
01360     S_predicted += V_predicted*(V_predicted.transpose());
01361     }
01362     S_predicted /= ((double)(Nparticles));
01363     V_predicted = Y - Y_predicted;
01364     for(int i=0; i < V_predicted.size(); ++i){
01365         if(nonlinear_model.Y_isangle[i]) V_predicted(i) =
atan2(sin(V_predicted(i)),cos(V_predicted(i)));
01366     }
01367     d2_predicted = (V_predicted.transpose() * S_predicted.inverse() * V_predicted)(0, 0);
01368
01369     // Resampling
01370     Eigen::VectorXd WparticlesCumSum(Nparticles);
01371     WparticlesCumSum(0) = Wparticles(0);
01372     for (long int i = 1; i < Nparticles; ++i) {
01373         WparticlesCumSum(i) = WparticlesCumSum(i-1) + Wparticles(i);
01374     }
01375     {
01376         long int i = 0;
01377         double u1 = UnivariateUniformSampler(0.0, 1.0)/((double)(Nparticles));
01378         for (long int j = 0; j < Nparticles; ++j) {
01379             double u = u1 + ((double)(j))/((double)(Nparticles));
01380             if (u > 1.0){ u = 1.0; }
01381             while (u > WparticlesCumSum(i)){
01382                 i = i + 1;
01383                 if (i > Nparticles-1){
01384                     i = Nparticles-1;
01385                     break;
01386                 }
01387             }
01388             Xparticles.col(j) = Xparticles.col(i);
01389         }
01390     }
01391     ResetWeights();
01392
01393     // Compute correction
01394     X = Eigen::VectorXd::Zero(Nx);
01395     switch (estimate_type){
01396     case EstimateType::ConditionalMean:
01397     {
01398         for (long int i = 0; i < Nparticles; ++i) {
01399             X += Wparticles(i)*Xparticles.col(i);
01400         }
01401     }
01402     break;
01403     case EstimateType::MaximumAPosteriori:
01404     {
01405         long int imax = 0;
01406         for (long int i = 1; i < Nparticles; ++i) {
01407             if (Wparticles(i) > Wparticles(imax)) imax = i;
01408         }
01409         X = Xparticles.col(imax);
01410     }
01411     break;
01412     }
01413
01414     P = Eigen::MatrixXd::Zero(Nx,Nx);
01415     for (long int i = 0; i < Nparticles; ++i) {
01416         Eigen::VectorXd dX = Xparticles.col(i)-X;
01417         for(int j=0; j < dX.rows(); ++j){
01418             if(nonlinear_model.X_isangle[j]) dX(j) = atan2(sin(dX(j)),cos(dX(j)));
01419         }
01420         P += Wparticles(i)*(dX)*(dX.transpose());
01421     }
01422
01423     // Compute corrected output, innovation and d^2:
01424     Y_corrected = Eigen::VectorXd::Zero(Ny); // Computer with weights equal, leading to the
mean
01425     for (long int i = 0; i < Nparticles; ++i) {
01426         Xi = Xparticles.col(i);
01427         res = MultivariateZeroMeanGaussianSampler(Vi, nonlinear_model.R); if(!res.first)
return res;
01428         res = nonlinear_model.MeasurementModel(Yi,Xi,t,Vi); if(!res.first) return res;
01429         Y_corrected += Yi; // Simple mean
01430         Yparticles.col(i) = Yi;
01431     }
01432     Y_corrected /= ((double)(Nparticles));
01433     S_corrected = Eigen::MatrixXd::Zero(Ny,Ny);
01434     for (int i = 0; i < Nparticles; ++i) {
01435         V_corrected = Yparticles.col(i)-Y_corrected;
01436         for(int j=0; j < V_corrected.size(); ++j){
01437             if(nonlinear_model.Y_isangle[j]) V_corrected(j) =
atan2(sin(V_corrected(j)),cos(V_corrected(j)));
01438         }
01439         S_corrected += (V_corrected)*(V_corrected.transpose());

```

```
01440         }
01441         S_corrected /= ((double) (Nparticles));
01442         V_corrected = Y - Y_corrected;
01443         for(int i=0; i < V_corrected.size(); ++i){
01444             if(nonlinear_model.Y_isangle[i]) V_corrected(i) =
01445             atan2(sin(V_corrected(i)),cos(V_corrected(i)));
01446             d2_corrected = (V_corrected.transpose() * S_corrected.inverse() * V_corrected)(0, 0);
01447         }
01448         return res;
01449     }
01450 };
01451 };
01452 }
01453
01454 #endif // BayesianFiltering_H
```

8.3 README.md File Reference

Index

- ~AdaptiveKalmanFilter
 - BayesianFiltering::AdaptiveKalmanFilter, [20](#)
- ~BayesianFilter
 - BayesianFiltering::BayesianFilter, [23](#)
- ~ExtendedKalmanFilter
 - BayesianFiltering::ExtendedKalmanFilter, [27](#)
- ~KalmanFilter
 - BayesianFiltering::KalmanFilter, [31](#)
- ~LinearBayesianFilter
 - BayesianFiltering::LinearBayesianFilter, [34](#)
- ~NonlinearBayesianFilter
 - BayesianFiltering::NonlinearBayesianFilter, [37](#)
- ~RobustSelfAdaptiveKalmanFilter
 - BayesianFiltering::RobustSelfAdaptiveKalmanFilter, [40](#)
- ~SamplingImportanceResamplingFilter
 - BayesianFiltering::SamplingImportanceResamplingFilter, [45](#)
- ~StateSpaceNonlinearModelBase
 - BayesianFiltering::StateSpaceNonlinearModelBase, [54](#)
- ~UnscentedKalmanFilter
 - BayesianFiltering::UnscentedKalmanFilter, [61](#)
- A
 - BayesianFiltering::StateSpaceLinearModel, [50](#)
- About this library, [1](#)
- alpha
 - BayesianFiltering::AdaptiveKalmanFilter, [21](#)
- B
 - BayesianFiltering::StateSpaceLinearModel, [50](#)
- BayesianFilter
 - BayesianFiltering::BayesianFilter, [23](#)
- BayesianFiltering, [13](#)
 - MultivariateGaussianSampler, [14](#)
 - MultivariateZeroMeanGaussianSampler, [14](#)
 - Result, [14](#)
 - ResultFormat, [15](#)
 - UnivariateGaussianSampler, [15](#)
 - UnivariateUniformSampler, [16](#)
- bayesianfiltering.hpp, [65](#)
 - PRINT_EIGEN_MATRIX, [67](#)
 - PRINT_EIGEN_VECTOR, [67](#)
 - PRINT_LINE_DEBUG, [67](#)
 - PrintEigenMatrixXd, [68](#)
 - PrintEigenVectorXd, [68](#)
- BayesianFiltering::AdaptiveKalmanFilter, [17](#)
 - ~AdaptiveKalmanFilter, [20](#)
 - alpha, [21](#)
 - beta, [21](#)
 - Correction, [20](#)
 - Prediction, [20](#)
 - Reset, [20, 21](#)
- BayesianFiltering::BayesianFilter, [22](#)
 - ~BayesianFilter, [23](#)
 - BayesianFilter, [23](#)
 - d2_corrected, [24](#)
 - d2_predicted, [24](#)
 - P, [24](#)
 - Reset, [23](#)
 - S_corrected, [24](#)
 - S_predicted, [24](#)
 - V_corrected, [24](#)
 - V_predicted, [24](#)
 - X, [25](#)
 - Y_corrected, [25](#)
 - Y_predicted, [25](#)
- BayesianFiltering::ExtendedKalmanFilter, [25](#)
 - ~ExtendedKalmanFilter, [27](#)
 - Correction, [27](#)
 - Prediction, [28](#)
 - Reset, [28](#)
- BayesianFiltering::KalmanFilter, [29](#)
 - ~KalmanFilter, [31](#)
 - Correction, [31](#)
 - Prediction, [31](#)
 - Reset, [32](#)
- BayesianFiltering::LinearBayesianFilter, [32](#)
 - ~LinearBayesianFilter, [34](#)
 - CheckModel, [34](#)
 - Correction, [34](#)
 - Prediction, [34](#)
 - Reset, [34](#)
- BayesianFiltering::NonlinearBayesianFilter, [35](#)
 - ~NonlinearBayesianFilter, [37](#)
 - Correction, [37](#)
 - Prediction, [37](#)
 - Reset, [37](#)
- BayesianFiltering::RobustSelfAdaptiveKalmanFilter, [38](#)
 - ~RobustSelfAdaptiveKalmanFilter, [40](#)
 - Correction, [40](#)
 - Prediction, [40](#)
 - Reset, [41](#)
- BayesianFiltering::SamplingImportanceResamplingFilter, [41](#)
 - ~SamplingImportanceResamplingFilter, [45](#)
 - ComputeESS, [45](#)
 - ConditionalMean, [45](#)

- Correction, [45](#)
- EnumToString, [45](#)
- ESS, [47](#)
- estimate_type, [47](#)
- EstimateType, [44](#)
- MaximumAPosteriori, [45](#)
- NormalizeWeights, [45](#)
- Nparticles, [47](#)
- Prediction, [46](#)
- Reset, [46](#)
- Wparticles, [47](#)
- Xparticles, [47](#)
- Yparticles, [47](#)
- BayesianFiltering::StateSpaceLinearModel, [48](#)
 - A, [50](#)
 - B, [50](#)
 - C, [50](#)
 - MeasurementModel, [49](#)
 - Nu, [50](#)
 - Nv, [51](#)
 - Nw, [51](#)
 - Nx, [51](#)
 - Ny, [51](#)
 - P0, [51](#)
 - ProcessModel, [49](#)
 - Q, [51](#)
 - R, [51](#)
 - SetDimensions, [50](#)
 - X0, [51](#)
 - X_isangle, [52](#)
 - Y_isangle, [52](#)
- BayesianFiltering::StateSpaceNonlinearModelBase, [52](#)
 - ~StateSpaceNonlinearModelBase, [54](#)
 - dfdW, [57](#)
 - dfdX, [57](#)
 - dhdV, [57](#)
 - dhdX, [57](#)
 - MeasurementModel, [54](#)
 - Nu, [57](#)
 - Nv, [57](#)
 - Nw, [57](#)
 - Nx, [57](#)
 - Ny, [58](#)
 - P0, [58](#)
 - ProcessModel, [54](#)
 - Q, [58](#)
 - R, [58](#)
 - SetDimensions, [55](#)
 - UpdatedfdW, [55](#)
 - UpdatedfdX, [55](#)
 - UpdatedhdV, [56](#)
 - UpdatedhdX, [56](#)
 - X0, [58](#)
 - X_isangle, [58](#)
 - Y_isangle, [58](#)
- BayesianFiltering::UnscentedKalmanFilter, [59](#)
 - ~UnscentedKalmanFilter, [61](#)
 - Correction, [62](#)
 - Prediction, [63](#)
 - Reset, [63](#)
- beta
 - BayesianFiltering::AdaptiveKalmanFilter, [21](#)
- C
 - BayesianFiltering::StateSpaceLinearModel, [50](#)
- CheckModel
 - BayesianFiltering::LinearBayesianFilter, [34](#)
- ComputeESS
 - BayesianFiltering::SamplingImportanceResamplingFilter, [45](#)
- ConditionalMean
 - BayesianFiltering::SamplingImportanceResamplingFilter, [45](#)
- Correction
 - BayesianFiltering::AdaptiveKalmanFilter, [20](#)
 - BayesianFiltering::ExtendedKalmanFilter, [27](#)
 - BayesianFiltering::KalmanFilter, [31](#)
 - BayesianFiltering::LinearBayesianFilter, [34](#)
 - BayesianFiltering::NonlinearBayesianFilter, [37](#)
 - BayesianFiltering::RobustSelfAdaptiveKalmanFilter, [40](#)
 - BayesianFiltering::SamplingImportanceResamplingFilter, [45](#)
 - BayesianFiltering::UnscentedKalmanFilter, [62](#)
- d2_corrected
 - BayesianFiltering::BayesianFilter, [24](#)
- d2_predicted
 - BayesianFiltering::BayesianFilter, [24](#)
- dfdW
 - BayesianFiltering::StateSpaceNonlinearModelBase, [57](#)
- dfdX
 - BayesianFiltering::StateSpaceNonlinearModelBase, [57](#)
- dhdV
 - BayesianFiltering::StateSpaceNonlinearModelBase, [57](#)
- dhdX
 - BayesianFiltering::StateSpaceNonlinearModelBase, [57](#)
- EnumToString
 - BayesianFiltering::SamplingImportanceResamplingFilter, [45](#)
- ESS
 - BayesianFiltering::SamplingImportanceResamplingFilter, [47](#)
- estimate_type
 - BayesianFiltering::SamplingImportanceResamplingFilter, [47](#)
- EstimateType
 - BayesianFiltering::SamplingImportanceResamplingFilter, [44](#)
- MaximumAPosteriori

- BayesianFiltering::SamplingImportanceResamplingFilter,
 - PRINT_LINE_DEBUG, bayesianfiltering.hpp, 67
- MeasurementModel
 - BayesianFiltering::StateSpaceLinearModel, 49
 - BayesianFiltering::StateSpaceNonlinearModelBase, 54
- MultivariateGaussianSampler
 - BayesianFiltering, 14
- MultivariateZeroMeanGaussianSampler
 - BayesianFiltering, 14
- NormalizeWeights
 - BayesianFiltering::SamplingImportanceResamplingFilter, 45
- Nparticles
 - BayesianFiltering::SamplingImportanceResamplingFilter, 47
- Nu
 - BayesianFiltering::StateSpaceLinearModel, 50
 - BayesianFiltering::StateSpaceNonlinearModelBase, 57
- Nv
 - BayesianFiltering::StateSpaceLinearModel, 51
 - BayesianFiltering::StateSpaceNonlinearModelBase, 57
- Nw
 - BayesianFiltering::StateSpaceLinearModel, 51
 - BayesianFiltering::StateSpaceNonlinearModelBase, 57
- Nx
 - BayesianFiltering::StateSpaceLinearModel, 51
 - BayesianFiltering::StateSpaceNonlinearModelBase, 57
- Ny
 - BayesianFiltering::StateSpaceLinearModel, 51
 - BayesianFiltering::StateSpaceNonlinearModelBase, 58
- P
 - BayesianFiltering::BayesianFilter, 24
- P0
 - BayesianFiltering::StateSpaceLinearModel, 51
 - BayesianFiltering::StateSpaceNonlinearModelBase, 58
- Prediction
 - BayesianFiltering::AdaptiveKalmanFilter, 20
 - BayesianFiltering::ExtendedKalmanFilter, 28
 - BayesianFiltering::KalmanFilter, 31
 - BayesianFiltering::LinearBayesianFilter, 34
 - BayesianFiltering::NonlinearBayesianFilter, 37
 - BayesianFiltering::RobustSelfAdaptiveKalmanFilter, 40
 - BayesianFiltering::SamplingImportanceResamplingFilter, 46
 - BayesianFiltering::UnscentedKalmanFilter, 63
- PRINT_EIGEN_MATRIX
 - bayesianfiltering.hpp, 67
- PRINT_EIGEN_VECTOR
 - bayesianfiltering.hpp, 67
- ProcessModel
 - BayesianFiltering::StateSpaceLinearModel, 49
 - BayesianFiltering::StateSpaceNonlinearModelBase, 54
- Q
 - BayesianFiltering::StateSpaceLinearModel, 51
 - BayesianFiltering::StateSpaceNonlinearModelBase, 58
- R
 - BayesianFiltering::StateSpaceLinearModel, 51
 - BayesianFiltering::StateSpaceNonlinearModelBase, 58
- README.md, 82
- Reset
 - BayesianFiltering::AdaptiveKalmanFilter, 20, 21
 - BayesianFiltering::BayesianFilter, 23
 - BayesianFiltering::ExtendedKalmanFilter, 28
 - BayesianFiltering::KalmanFilter, 32
 - BayesianFiltering::LinearBayesianFilter, 34
 - BayesianFiltering::NonlinearBayesianFilter, 37
 - BayesianFiltering::RobustSelfAdaptiveKalmanFilter, 41
 - BayesianFiltering::SamplingImportanceResamplingFilter, 46
 - BayesianFiltering::UnscentedKalmanFilter, 63
- Result
 - BayesianFiltering, 14
- ResultFormat
 - BayesianFiltering, 15
- S_corrected
 - BayesianFiltering::BayesianFilter, 24
- S_predicted
 - BayesianFiltering::BayesianFilter, 24
- SetDimensions
 - BayesianFiltering::StateSpaceLinearModel, 50
 - BayesianFiltering::StateSpaceNonlinearModelBase, 55
- UnivariateGaussianSampler
 - BayesianFiltering, 15
- UnivariateUniformSampler
 - BayesianFiltering, 16
- UpdatedfdW
 - BayesianFiltering::StateSpaceNonlinearModelBase, 55
- UpdatedfdX
 - BayesianFiltering::StateSpaceNonlinearModelBase, 55
- UpdatedhdV

- BayesianFiltering::StateSpaceNonlinearModelBase,
[56](#)
- UpdatedhdX
 - BayesianFiltering::StateSpaceNonlinearModelBase,
[56](#)
- V_corrected
 - BayesianFiltering::BayesianFilter, [24](#)
- V_predicted
 - BayesianFiltering::BayesianFilter, [24](#)
- Wparticles
 - BayesianFiltering::SamplingImportanceResamplingFilter,
[47](#)
- X
 - BayesianFiltering::BayesianFilter, [25](#)
- X0
 - BayesianFiltering::StateSpaceLinearModel, [51](#)
 - BayesianFiltering::StateSpaceNonlinearModelBase,
[58](#)
- X_isangle
 - BayesianFiltering::StateSpaceLinearModel, [52](#)
 - BayesianFiltering::StateSpaceNonlinearModelBase,
[58](#)
- Xparticles
 - BayesianFiltering::SamplingImportanceResamplingFilter,
[47](#)
- Y_corrected
 - BayesianFiltering::BayesianFilter, [25](#)
- Y_isangle
 - BayesianFiltering::StateSpaceLinearModel, [52](#)
 - BayesianFiltering::StateSpaceNonlinearModelBase,
[58](#)
- Y_predicted
 - BayesianFiltering::BayesianFilter, [25](#)
- Yparticles
 - BayesianFiltering::SamplingImportanceResamplingFilter,
[47](#)